

آموزش ریکت

فهرست

آموزش ریکت	۱
فصل اول- مبانی	۳
۱. مقدمه و معرفی دوره	۳
جمع‌بندی و مسیر یادگیری	۵
مراحل ایجاد پروژه	۹
آشنایی با ساختار پروژه ایجاد شده توسط <i>Vite</i>	۱۰
۳. اجرای اولین برنامه <i>React</i>	۱۳
فصل دوم: پایه‌های <i>React</i> و <i>JSX</i> (۲ جلسه)	۱۳
مفهوم کامپوننت (<i>Component</i>)	۱۳
فلسفه <i>Component-Based Architecture</i>	۱۳
تفاوت کامپوننت‌های <i>Class</i> و <i>Functional</i>	۱۶
<i>JSX (JavaScript XML)</i>	۱۹
۳. نوشتن <i>Expression</i> در <i>JSX</i> با <code>{}</code>	۲۰
کار با رویدادها (<i>Events</i>)	۲۱
<i>Conditional Rendering</i> (رندر شرطی)	۲۳
۶. رندر لیست‌ها و استفاده از <code>key</code>	۲۵
۱. الگوریتم <i>Diffing</i> (مقایسه)	۲۷
۲. حفظ وضعیت (<i>State</i>)	۲۷
فصل سوم: مدیریت <i>State</i> و <i>Props</i> (۲ جلسه)	۳۲
مفهوم <i>Props</i> : ارسال داده از <i>Parent</i> به <i>Child</i>	۳۲

۳۵	۲. بررسی نوع داده (<i>Type Checking</i>)
۳۸	آموزش گام به گام: ایجاد کامپوننت <i>ProductList`</i>
۴۲	گام ۲: ایجاد و تعریف کامپوننت فرزند (<i>ProductList`</i>)
۴۴	گام ۳: فراخوانی کامپوننت فرزند و ارسال داده‌ها (<i>Props</i>)
۴۵	مفهوم جریان داده یک‌طرفه (<i>Unidirectional Data Flow</i>)
۴۶	جداسازی کامپوننت‌ها در فایل‌های مجزا
۵۰	<i>State</i> در ریکت چیست و چه کاربردی دارد:
۵۶	آموزش مدیریت فرم و لیست کاربران در <i>React</i>
۵۹	فصل چهارم: هوک‌های حیاتی و اثرات جانبی
۶۰	۱. هوک <i>useEffect</i> : مدیریت اثرات جانبی
۶۲	کاربرد <i>useEffect</i> در خواندن اطلاعات
۶۲	آموزش اتصال <i>React</i> به <i>WordPress</i> با استفاده از <i>useEffect</i>
۶۳	مرحله ۲: فراخوانی در فایل اصلی (<i>App.jsx</i>)
۶۵	بکارگیری <i>useEffect</i> در تعامل با دنیای خارج از <i>React (Direct DOM Manipulation)</i>
۶۷	بکارگیری <i>useEffect</i> ساخت یک ساعت آنالوگ
۷۰	فصل پنجم: مدیریت <i>State</i>
۷۰	یادآوری <i>useState</i> :
۷۱	تابع پاکسازی (<i>Cleanup Function</i>):
۷۳	۲- سایر کامپوننت‌ها
۷۸	مثال ایجاد تم شب
۸۶	فصل ششم: مسیریابی (<i>Routing</i>) با <i>React Router</i>
۸۷	نصب و راه‌اندازی <i>React Router DOM</i>
۸۷	ایجاد روت‌های اصلی

۹۳.....	۳. مسیریابی پویا
۹۶.....	لینک‌دهی و <i>Navigation</i>
۹۹.....	فصل هفتم: کار با فرم‌ها و مدیریت آن‌ها
۱۰۰.....	فصل هشتم: بهینه‌سازی و ساخت (<i>Build</i>) (۱ جلسه)
۱۰۰.....	فصل نهم: تست‌نویسی (<i>Testing</i>) (۱ جلسه)
۱۰۱.....	فصل دهم: پروژه نهایی و جمع‌بندی (۲ جلسه)

فصل اول- مبانی

فصل اول: مبانی و آماده‌سازی محیط توسعه (۱ جلسه)

هدف: دانشجو بتواند محیط توسعه حرفه‌ای را راه‌اندازی کرده و اولین برنامه *React* خود را با *Vite* اجرا کند.

۱. مقدمه و معرفی دوره

در این دوره جامع، *React* را به صورت پروژه‌محور و با استفاده از جدیدترین نسخه این کتابخانه فرا خواهید گرفت. با ابزار مدرن *Vite* کار خواهیم کرد تا تجربه توسعه سریع و بهینه‌ای داشته باشید. از مبانی اولیه تا مفاهیم پیشرفته را پوشش می‌دهیم و در پایان دوره قادر خواهید بود اپلیکیشن‌های حرفه‌ای *React* ایجاد کنید. این دوره برای توسعه‌دهندگان *JavaScript* که می‌خواهند اصولی و کاربردی *React* را بیاموزند طراحی شده است.

چرا React؟ مقایسه با سایر کتابخانه‌ها/فریمورک‌ها

React امروزه به یکی از محبوب‌ترین کتابخانه‌های فرانت‌اند تبدیل شده. اما چرا؟

در مقایسه با *Angular*:

- *React* یادگیری ساده‌تری دارد و انعطاف‌پذیرتر است

- **Angular** یک فریمورک کامل است، در حالی که **React** یک کتابخانه است

- **React** جامعه بزرگتر و اکوسیستم غنی‌تری دارد

در مقایسه با **Vue**:

- هر دو یادگیری نسبتاً ساده‌ای دارند

- **React** بازار کار گسترده‌تری در ایران و جهان دارد

- **JSX** در **React vs Template Syntax** در **Vue**

مزایای اصلی **React**:

- قابل استفاده در وب، موبایل (**React Native**) و دسکتاپ

- جامعه فعال و کامپوننت‌های آماده فراوان

- مناسب برای پروژه‌های کوچک تا بسیار بزرگ

- پشتیبانی قوی توسط فیسبوک

پس **React** انتخاب بهتری است اگر:

- می‌خواهید بازار کار بهتری داشته باشید

- به انعطاف‌پذیری بیشتر نیاز دارید

- قصد دارید در پلتفرم‌های مختلف توسعه دهید

• معرفی قابلیت‌های کلیدی و جدید React

ری‌اکت با قابلیت‌های جدیدی مانند کامپوننت **Activity**، هوک **useEffectEvent** و بهبودهای عملکردی برای توسعه‌دهندگان منتشر شده است. جدول زیر خلاصه‌ای از مهم‌ترین ویژگی‌های این نسخه را نشان می‌دهد.

ویژگی	دسته‌بندی	توضیح مختصر
-------	-----------	-------------

مدیریت خودکار رندها و حذف نیاز به <code>useMemo</code> <code>useCallback</code> و <code>memo`</code> .	بهینه‌سازی عملکرد	کامپایلر ری‌اکت (<i>React Forget</i>)
اجرای کامپوننت‌ها در سرور برای بهبود سئو و عملکرد.	معماری	کامپوننت‌های سمت سرور
مدیریت یکپارچه و ساده‌تر فرم‌ها در سمت کلاینت و سرور.	تجربه توسعه‌دهنده (<i>DX</i>)	<i>Actions</i>
حل مشکل " <i>Closures</i> " قدیمی" در هوک <code>useEffect</code> بدون وابستگی اضافه.	تجربه توسعه‌دهنده (<i>DX</i>)	هوک <code>useEffectEvent`</code>
مخفی کردن بخشی از <i>UI</i> بدون از دست دادن حالت (<i>State</i>) آن.	عملکرد و <i>UX</i>	کامپوننت <code></ Activity></code>
پیش‌رندر بخش‌های استاتیک برنامه و هیدراته کردن پویا پس از آن.	عملکرد	رندر جزئی (<i>Partial Pre-rendering</i>)
استفاده راحت‌تر از وب‌کامپوننت‌ها درون برنامه‌های ری‌اکت.	یکپارچه‌سازی	وب‌کامپوننت‌ها
مدیریت مستقیم تگ‌های متا (مانند <code><title></code>) درون <i>JSX</i> .	تجربه توسعه‌دهنده (<i>DX</i>)	مدیریت <i>Metadata</i> سند

جمع‌بندی و مسیر یادگیری

همان‌طور که مشاهده کردید، ری‌اکت تمرکز زیادی بر بهبود عملکرد برنامه (با ویژگی‌هایی مانند کامپایلر و کامپوننت `Activity``) و تجربه توسعه‌دهنده (با هوک‌هایی مانند `useEffectEvent`` و `Actions``) دارد.

چرا Vite؟ مقایسه با Create React App (CRA) از نظر سرعت و کارایی

انتخاب `Vite` به جای `Create React App` (CRA) عمدتاً به دلیل سرعت بسیار بالاتر در محیط توسعه و ساخت نهایی است. این تفاوت سرعت ناشی از معماری مدرن `Vite` است که از ماژول‌های ES بومی مرورگر استفاده می‌کند، در حالی که `CRA` از سیستم باندلینگ سنتی `Webpack` بهره می‌برد. در ادامه، مقایسه دقیق‌تری از جنبه‌های مختلف این دو ابزار ارائه شده است:

ویژگی	Vite	Create React App (CRA)
معماری و سرعت توسعه	استفاده از ES Modules بومی مرورگر؛ سرور توسعه در کمتر از ۲ ثانیه راه‌اندازی می‌شود.	استفاده از Webpack و باندلینگ کامل؛ راه‌اندازی سرور توسعه کند (حتی تا ۶۰ ثانیه).
Hot Module Replacement	HMR بسیار سریع و تقریباً آنی؛ تغییرات بدون رفرش صفحه اعمال می‌شود.	HMR کندتر؛ اغلب نیاز به رفرش کامل صفحه دارد.
ساخت نهایی	استفاده از Rollup برای ساخت نهایی؛ زمان بسیار کوتاه‌تر و خروجی بهینه‌تر.	استفاده از Webpack برای ساخت؛ زمان طولانی‌تر و حجم خروجی بیشتر.
پیکربندی	پیکربندی ساده و قابل گسترش با فایل vite.config.js و اکوسیستم پلاگین غنی.	پیکربندی محدود؛ برای سفارشی‌سازی نیاز به Eject کردن و خروج از حالت امن دارد.

پشتیبانی و آینده	جامعه کاربری در حال رشد سریع، پشتیبانی از استانداردهای مدرن و توصیه شده توسط مستندات رسمی ری اکت.	جامعه بزرگ اما ثابت؛ دیگر توسط تیم ری اکت پشتیبانی و برای پروژه‌های جدید توصیه نمی‌شود.
------------------	---	--

جمع‌بندی نهایی

Vite را انتخاب کنید اگر:

- سرعت توسعه برای شما اولویت دارد.
- قصد دارید روی پروژه‌های مدرن و مقیاس‌پذیر کار کنید.
- به پیکربندی انعطاف‌پذیر بدون نیاز به *Eject* کردن علاقه‌مند هستید.

Create React App ممکن است برای موارد زیر مناسب‌تر باشد:

- تازه‌کارهایی که به سادگی و راه‌اندازی سریع (بدون در نظر گرفتن سرعت توسعه) نیاز دارند.
- کار روی پروژه‌های کوچک که سرعت توسعه در آن‌ها چالش اصلی نیست.

۲. راه‌اندازی محیط توسعه

۱. نصب *Node.js*

- به سایت رسمی (<https://nodejs.org>) [nodejs.org] مراجعه کنید
- نسخه *LTS* (نسخه پایدار) را دانلود و نصب کنید
- نسخه *LTS* برای اکثر کاربران توصیه می‌شود

۲. *NPM* چیست؟

- *NPM (Node Package Manager)* یک ابزار مدیریت پکیج است
- همراه *Node.js* به صورت خودکار نصب می‌شود

- برای نصب کتابخانه‌ها و ابزارهای مورد نیاز پروژه استفاده می‌شود

۳. بررسی نصب صحیح *Node.js* و *NPM*

- ترمینال (*Terminal*) سیستم خود را باز کنید

- دستورات زیر را وارد کنید:

```
node --version
```

(بررسی نسخه *Node.js* - باید شماره نسخه نمایش داده شود)

```
npm --version
```

(بررسی نسخه *NPM* - باید شماره نسخه نمایش داده شود)

در صورت بروز خطا و عدم یافتن *NPM* باید نود را نصب کنید.

نصب *Node.js* و *npm*/*yarn*/*pnpm*

نصب *Node.js*

- به سایت (<https://nodejs.org>) [nodejs.org] مراجعه کنید

- نسخه *LTS (Recommended)* را دانلود و نصب کنید

- این نسخه پایدار بوده و برای اکثر کاربران مناسب است

Yarn و *PNPM* چیستند؟

- *Yarn*: مدیر پکیج جایگزین که توسط فیسبوک توسعه یافته

- *PNPM*: مدیر پکیج سریع‌تر و بهینه‌تر با مصرف فضای کمتر

نصب *Yarn* و *PNPM* (اختیاری)

```
npm install -g yarn  
npm install -g pnpm
```

ایجاد اولین پروژه

پیش‌نیازها

- *Node.js* نسخه ۱۸ یا بالاتر

- *NPM* (که همراه *Node.js* نصب می‌شود)

مراحل ایجاد پروژه

۱. ایجاد پروژه جدید

یک پوشه ایجاد و آن را در ویژوال استودیو کد باز کنید

دستور زیر را در ترمینال اجرا کنید:

```
npm init -y
npm create vite@latest my-app -- --template react-ts
or
npm create vite@latest my-app -- --template react
```

۲. پیکربندی پروژه

- نام پروژه را وارد کنید

- از بین فریمورک‌ها، *React* را انتخاب کنید

- از بین انواع پروژه، *JavaScript* یا *TypeScript* را انتخاب کنید

۳. نصب *dependencies*

```
cd project-name
npm install
```

۴. ارتقاء به *React* آخرین نسخه

```
npm install react@latest react-dom@latest
```

۵. اجرای پروژه

```
npm run dev
```

برای اجرای نمونه کدهایی که همراه آموزش وجود دارد پوشه آموزش را در نرم افزار *visual studio code* باز کنید. سپس با کمک کلید میانبر **CTRL+** یا از منوی **View/terminal** را انتخاب کنید سپس دستور

```
Cd vite-project
```

سپس دستور

```
npm install
```

و بعد دستور

```
npm run dev
```

ساختار پروژه

- **src**: کدهای اصلی پروژه

- **public**: فایل‌های استاتیک

- **index.html**: فایل **HTML** اصلی

- **main.jsx**: نقطه ورود برنامه

- **App.jsx**: کامپوننت اصلی

- **package.json**: تنظیمات پروژه

تست پروژه

پس از اجرای دستور **npm run dev**، مرورگر را باز کرده و به آدرس **http://localhost:5173** بروید. باید صفحه خوش‌آمدگویی **React** را ببینید.

اولین تغییر

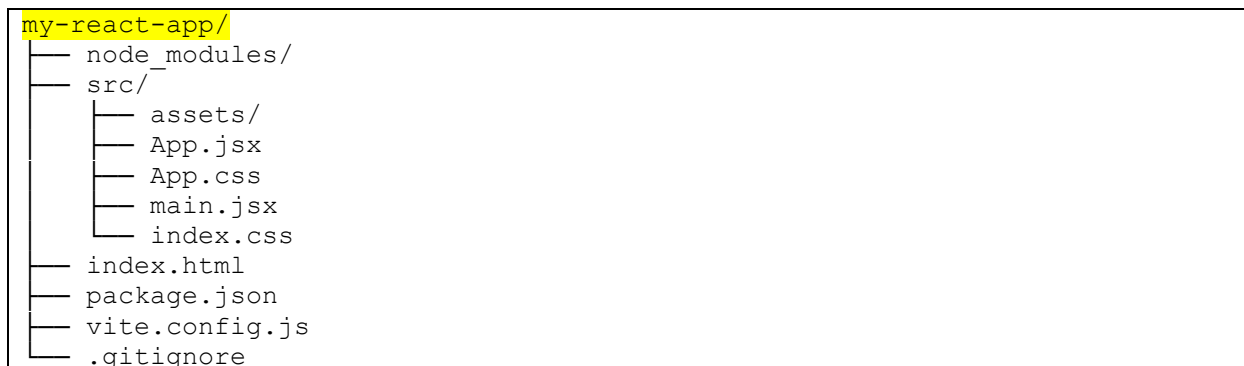
فایل **App.jsx** را باز کرده و محتوای آن را تغییر دهید. تغییرات به صورت خودکار در مرورگر نمایش داده می‌شود.

آشنایی با ساختار پروژه ایجاد شده توسط Vite

ساختار پروژه ایجادشده توسط **Vite** بسیار ساده و منطقی طراحی شده تا بتوانید بلافاصله کار توسعه را شروع کنید. در ادامه با اجزای اصلی این ساختار آشنا می‌شوید.

بررسی ساختار پروژه

پس از ایجاد یک پروژه **React** با **Vite**، ساختار فایل‌های شما به شکل زیر خواهد بود:



توضیح کارکرد فایل‌های کلیدی

در اینجا نقش و کاربرد هر یک از فایل‌های اصلی را بررسی می‌کنیم:

فایل/پوشه	توضیح
<code>index.html</code>	فایل HTML اصلی و نقطه ورود برنامه. Vite به طور مستقیم این فایل را در حین توسعه سرو می‌دهد.
<code>src/main.jsx</code>	نقطه شروع برنامه React . کامپوننت اصلی <code>App</code> را در المنت <code>root</code> موجود در <code>index.html</code> رندر می‌کند.

<code>`src/App.jsx`</code>	کامپوننت اصلی برنامه. توسعه برنامه معمولاً با تغییر و گسترش این فایل و ساخت کامپوننت‌های جدید شروع می‌شود.
<code>`/src/assets`</code>	مخصوص فایل‌های رسانه مانند عکس‌ها و آیکون‌ها.
<code>`package.json`</code>	شامل لیست کتابخانه‌های مورد استفاده و اسکریپت‌های پروژه (مانند <code>`dev`</code> ، <code>`build`</code> ، <code>`preview`</code>).
<code>`vite.config.js`</code>	فایل پیکربندی Vite برای سفارشی‌سازی تنظیمات و افزودن پلاگین‌ها.

نکات کاربردی برای شروع

- تغییرات سریع: هنگام ذخیره فایل‌هایی مانند ``App.jsx``، **Vite** تغییرات را به صورت لحظه‌ای و بدون نیاز به رفرش کامل صفحه در مرورگر نمایش می‌دهد. این قابلیت به دلیل **Hot Module Replacement** یا **HMR** است.

- مدیریت استایل: فایل‌های **CSS** مربوط به هر کامپوننت را می‌توانید در کنار فایل **JSX** آن قرار دهید و مستقیماً **import** کنید.

- ساخت نسخه نهایی: برای آماده‌سازی پروژه برای انتشار، از دستور ``npm run build`` استفاده کنید. خروجی نهایی در پوشه ``dist`` قرار می‌گیرد.

این ساختار تمیز و قابل درک، پایه محکمی برای ساختن برنامه‌های بزرگ و پیچیده‌تر فراهم می‌کند. معرفی پلاگین‌های مفید *Vite* (مانند *React Refresh*)

۳. اجرای اولین برنامه *React*

* دستور `npm run dev` برای اجرای سرور توسعه

* دستور `npm run build` برای ساخت نسخه *Production*

فصل دوم: پایه‌های *React* و *JSX* (۲ جلسه)

هدف: درک مفهوم کامپوننت و تسلط کامل بر سینتکس *JSX* و نحوه رندر کردن اطلاعات.

۱. مفهوم کامپوننت (*Component*)

* فلسفه *Component-Based Architecture*

* تفاوت کامپوننت‌های *Functional* و *Class* (ذکر مختصر با تأکید بر برتری *Functional*)

۲. *JSX (JavaScript XML)*

* قوانین نوشتن *JSX* (تگ بسته، یک المنت والد و ...)

* نوشتن *Expression* در *JSX* با `{}`

* کار با رویدادها (*Events*) مانند `onClick` و `onChange`

* کار با *Conditional Rendering* (رندر شرطی) با `&&` و *Ternary Operator*

* رندر لیست‌ها و استفاده از `key`

مفهوم کامپوننت (*Component*)

کامپوننت‌ها (*Components*) در ری‌اکت (*React*)، بلوک‌های ساختاری اصلی برای رابط کاربری (*UI*) هستند که به صورت توابع جاوااسکریپت ساده یا کلاس‌ها تعریف می‌شوند و منطق و ظاهر بخش کوچکی از *UI* را در بر می‌گیرند و خروجی آن‌ها معمولاً کد *JSX* است که نحوه‌ی نمایش آن بخش را مشخص می‌کند.

فلسفه *Component-Based Architecture*

معماری کامپوننت محور یک رویکرد طراحی نرم‌افزار است که در آن یک سیستم نرم‌افزاری بزرگ، از قطعات کوچک‌تر، مستقل و قابل استفاده مجدد به نام "کامپوننت" ساخته می‌شود. هر کامپوننت مانند یک بخش از کد اصلی (مشابه قطعات لگو) است که عملکرد خاصی را در قالب یک واحد کپسوله شده و با رابط‌های مشخص ارائه می‌دهد. این کامپوننت‌ها می‌توانند بدون تأثیرگذاری بر سایر بخش‌های سیستم، با یکدیگر ترکیب، جایگزین یا ارتقا داده شوند.

این مفهوم از دهه ۱۹۶۰ میلادی در متون آکادمیک مطرح شد، اما در دهه ۱۹۹۰ با ارائه مدل‌های تجاری مانند مدل شیء سیستمی **IBM (SOM)** و مدل شیء کامپوننت محور میکروسافت **(COM/OLE)** جنبه عملی‌تری به خود گرفت.

ویژگی‌های کلیدی یک کامپوننت خوب

یک کامپوننت ایده‌آل دارای ویژگی‌های زیر است:

ویژگی	توضیح
قابل استفاده مجدد	می‌تواند در بخش‌های مختلف یک برنامه یا حتی در پروژه‌های کاملاً متفاوت، بدون نیاز به تغییرات اساسی، مجدداً مورد استفاده قرار گیرد .
کپسوله شده	جزئیات پیاده‌سازی داخلی خود را پنهان می‌کند و تنها از طریق رابط‌های از پیش تعریف شده (Interfaces) با دنیای بیرون ارتباط برقرار می‌کند. این اصل، وابستگی بین کامپوننت‌ها را کاهش می‌دهد .
مستقل و دارای حداقل وابستگی	وابستگی کمی به کامپوننت‌های دیگر دارد و می‌تواند به صورت مستقل توسعه، آزمایش و استقرار یابد. تغییر در یک کامپوننت، تأثیر کمی بر کامپوننت‌های دیگر می‌گذارد .

قابل ویرایش	می‌توان آن را با کامپوننتی دیگر که همان رابط را پیاده‌سازی می‌کند، جایگزین کرد (مشروط بر حفظ قرارداد رابط) .
قابل ترکیب	کامپوننت‌ها را می‌توان برای ایجاد عملکردهای پیچیده‌تر و ساخت سیستم‌های بزرگ، در کنار یکدیگر قرار داد .

مزایای اصلی معماری کامپوننت محور

توسعه سریع‌تر و کاهش هزینه: با استفاده مجدد از کامپوننت‌های از پیش ساخته و آزمایش‌شده، نیاز به کدنویسی تکراری از بین می‌رود.

قابلیت نگهداری آسان‌تر: از آنجایی که هر کامپوننت مستقل است، رفع خطا یا افزودن قابلیت جدید به یک بخش، کل سیستم را تحت تأثیر قرار نمی‌دهد.

توسعه موازی: تیم‌های مختلف می‌توانند همزمان بر روی کامپوننت‌های متفاوت کار کنند و سرعت تحویل پروژه را افزایش دهند.

قابلیت اطمینان بالاتر: کامپوننت‌های قابل استفاده مجدد، معمولاً به دفعات بیشتری مورد آزمایش قرار گرفته‌اند، بنابراین پایدارتر و مطمئن‌تر هستند.

انعطاف‌پذیری و مقیاس‌پذیری: می‌توان کامپوننت‌ها را به صورت جداگانه ارتقا داد یا در صورت افزایش بار کاری، فقط کامپوننت‌های مورد نیاز را مقیاس کرد (افزودن نمونه‌های بیشتر)

چالش‌ها و ملاحظات

علیرغم مزایای زیاد، این معماری با چالش‌هایی نیز همراه است:

پیچیدگی مدیریت کامپوننت‌ها: در سیستم‌های بسیار بزرگ، مدیریت تعداد زیاد کامپوننت‌ها، وابستگی‌های بین آن‌ها، نسخه‌بندی و تضمین سازگاری می‌تواند دشوار باشد.

طراحی مرزهای کامپوننت: تعیین اندازه و حدود دقیق هر کامپوننت (نه خیلی ریز، نه خیلی درشت) نیاز به تفکر و طراحی دقیق دارد.

ایجاد بار ناخواسته اضافه: ارتباط بین کامپوننت‌ها (به ویژه اگر از طریق شبکه و در معماری‌های توزیع‌شده باشد) می‌تواند باعث ایجاد تأخیر (*Latency*) شود.

افزایش تلاش یکپارچه‌سازی: اطمینان از این که کامپوننت‌های مستقل ساخته‌شده، در نهایت به درستی با یکدیگر کار می‌کنند، نیاز به استراتژی‌های قوی یکپارچه‌سازی و آزمایش دارد.

مثال‌های عینی

فریم‌ورک‌های فرانت‌اند *React*، *Angular* و *Vue.js* نمونه‌های بارز این معماری هستند. در آن‌ها هر بخش از رابط کاربری (مثل دکمه، نوار جستجو، کارت محصول) یک کامپوننت مستقل است.

قابلیت جاسازی در نرم‌افزارها: امکان قرار دادن یک صفحه گسترده شبیه به *Excel* درون *PowerPoint*، نمونه‌ای کلاسیک از استفاده از یک کامپوننت (موتور پردازش صفحه‌گسترده) در یک برنامه دیگر است.

سیستم‌های سازمانی: در *Java EE/Jakarta EE*، کامپوننت‌هایی مانند *Enterprise JavaBeans (EJB)* برای پیاده‌سازی منطق کسب‌وکار استفاده می‌شوند.

تفاوت کامپوننت‌های *Class* و *Functional*

در *React* دو روش اصلی برای ساخت کامپوننت وجود دارد:

- کامپوننت‌های کلاسی (*Class Components*): روش مبتنی بر کلاس‌های جاوااسکریپت

- کامپوننت‌های تابعی (*Functional Components*): روش مبتنی بر توابع

برای درک عملی تفاوت ساختاری، هر دو نوع را ایجاد و اجرا می‌کنیم.

۲. ایجاد و اجرای کامپوننت کلاسی

مرحله ۱: ایجاد فایل کامپوننت کلاسی

در پوشه `src` پروژه *Vite* خود، فایل جدیدی با نام `TheClassComponent.jsx` ایجاد کنید و محتوای زیر را در آن قرار دهید:

```
// TheClassComponent.jsx
import React, { Component } from 'react';

class TheClassComponent extends Component {
  render() {
    return <h1>سلام! این یک کامپوننت کلاسی است</h1>;
  }
}
```



```
}  
}  
  
export default TheClassComponent;
```

مرحله ۲: فراخوانی در فایل اصلی

فایل `main.jsx` را باز کرده و محتوای آن را به صورت زیر ویرایش کنید:

```
// main.jsx  
import React from 'react'  
import ReactDOM from 'react-dom/client'  
import TheClassComponent from './TheClassComponent.jsx'  
  
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <TheClassComponent />  
  </React.StrictMode>,  
)
```

[src/components/TheClassComponent.jsx](#)

مرحله ۳: مشاهده نتیجه

سرور توسعه `Vite` را اجرا کنید (`npm run dev`) یا صفحه مرورگر را رفرش کنید. پیام "سلام! این یک کامپوننت کلاسی است." نمایش داده می‌شود.

۳. ایجاد و اجرای کامپوننت تابعی

مرحله ۱: ایجاد فایل کامپوننت تابعی

در پوشه `src`، فایل جدیدی با نام `TheFunctionalComponent.jsx` ایجاد کنید و محتوای زیر را در آن قرار دهید:

در نسخه تایپ اسکریپت نام فایل را `TheFunctionalComponent.tsx` می‌گذاریم.

```
// TheFunctionalComponent.jsx  
import React from 'react';  
  
function TheFunctionalComponent() {  
  return <h1>سلام! این یک کامپوننت تابعی است.</h1>;  
}  
  
export default TheFunctionalComponent;
```

مرحله ۲: فراخوانی در فایل اصلی

فایل `main.jsx` را به صورت زیر ویرایش کنید:

```
// main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import TheFunctionalComponent from './TheFunctionalComponent.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <TheFunctionalComponent />
  </React.StrictMode>,
)
```

مرحله ۳: مشاهده نتیجه

صفحه مرورگر را رفرش کنید. پیام "سلام! این یک کامپوننت تابعی است." نمایش داده می‌شود.

۴. تفاوت‌های ساختاری کلیدی

مشاهده اول: حجم و سادگی کد

- کامپوننت کلاسی نیاز به تعریف کلاس، ارث‌بری از *Component* و متد *render* دارد

- کامپوننت تابعی یک تابع ساده است که مستقیماً *JSX* برمی‌گرداند

مشاهده دوم: خوانایی

- کامپوننت تابعی ساختار مستقیم‌تر و قابل‌فهم‌تری دارد

- برای توسعه‌دهندگان تازه‌کار، درک کامپوننت تابعی آسان‌تر است

مشاهده سوم: استاندارد فعلی

- کامپوننت‌های تابعی روش استاندارد توسعه در *React* مدرن هستند

- تمام ویژگی‌های جدید *React* (مانند هوک‌ها) فقط برای کامپوننت‌های تابعی طراحی شده‌اند

۵. اجرای همزمان هر دو کامپوننت

ویرایش فایل *main.jsx* برای نمایش هر دو:

```
// main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import TheClassComponent from './TheClassComponent.jsx'
import TheFunctionalComponent from './TheFunctionalComponent.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <TheClassComponent />
  </React.StrictMode>
)
```

```
<TheFunctionalComponent />
</React.StrictMode>,
)
```

نتیجه نهایی: دو پیام جداگانه از دو نوع کامپوننت مختلف در صفحه نمایش داده می‌شود.

جمع‌بندی و نکات مهم

ویژگی	کامپوننت کلاسی	کامپوننت تابعی
ساختار	کلاس با متد <code>render</code>	تابع ساده
پیچیدگی	نیازمند آشنایی با <code>this</code>	بدون <code>this</code>
استاندارد	روش قدیمی	روش استاندارد فعلی
کاربرد	پروژه‌های قدیمی	تمام پروژه‌های جدید

JSX (JavaScript XML)

معرفی JSX

```
// مثال پایه JSX
const element = <h1>دنیا، سلام!</h1>;
```

قوانین اساسی نوشتن JSX

قانون ۱: تگ‌ها باید بسته شوند

تمام تگ‌های JSX باید به صورت کامل بسته شوند، حتی تگ‌های تکی (*self-closing*).

```
// صحیح
<div></div>
<input type="text" />


// غلط
<div> // ❌ تگ باز بدون بسته شدن
<br> // ❌ تگ تکی بدون بسته شدن
```

قانون ۲: فقط یک العنت والد

هر قطعه *JSX* باید یک العنت والد ریشه‌ای داشته باشد.

```
// صحیح - یک العنت والد
const element = (
  <div>
    <h۱>عنوان</h۱>
    <p>متن پاراگراف</p>
  </div>
);

// غلط - دو العنت هم‌سطح
const element = (
  <h۱>عنوان</h۱> // ❌
  <p>متن پاراگراف</p> // ❌
);
```

راه حل برای العنت‌های هم‌سطح: استفاده از *Fragment*

```
// صحیح - استفاده از Fragment
const element = (
  <>
    <h۱>عنوان</h۱>
    <p>متن پاراگراف</p>
  </>
);
```

۳. نوشتن *Expression* در *JSX* با `{ }`

می‌توانید کد جاوااسکریپت را درون *JSX* با استفاده از آکولاد `{ }` قرار دهید.

نمایش متغیرها

```
function Greeting() {
  const userName = "علی";
  const currentYear = ۲۰۲۴;

  return (
    <div>
      <h۱>سلام، {userName}!</h۱>
      <p>سال جاری: {currentYear}</p>
      <p>محاسبه: {۵ * ۱۰}</p>
    </div>
  );
}
```

```
    </div>
  );
}
```

فراخوانی توابع

```
function DisplayMessage() {
  const getGreeting = () => {
    const hour = new Date().getHours();
    if (hour < ۱۲) return "صبح بخیر";
    if (hour < ۱۸) return "ظهر بخیر";
    return "عصر بخیر";
  };

  return <p>{getGreeting()}</p>;
}
```

ویژگی‌های دینامیک (Dynamic Attributes)

```
function DynamicImage() {
  const imageUrl = "https://example.com/photo.jpg";
  const altText = "یک تصویر نمونه";

  return <img src={imageUrl} alt={altText} />;
}
```

کار با رویدادها (Events)

رویدادها در *JSX* با پیشوند *on* و با حروف بزرگ آغاز می‌شوند.

رویداد کلیک (*onClick*)

```
function ClickExample() {
  const handleClick = () => {
    console.log("دکمه کلیک شد");
  };

  return (
    <div>
      <button onClick={handleClick}>کلیک کن</button>

      {/* درجا arrow function با */}
      <button onClick={() => console.log("کلیک مستقیم")}>
        کلیک مستقیم
      </button>
    </div>
  );
}
```

```

    </button>
  </div>
);
}

```

رویداد تغییر (`onChange`)

```

function InputExample() {
  const handleChange = (event) => {
    console.log("مقدار جدید:", event.target.value);
  };

  return (
    <div>
      <input
        type="text"
        onChange={handleChange}
        placeholder="متن خود را وارد کنید"
      />
    </div>
  );
}

```

رویدادهای دیگر

```

function MultipleEvents() {
  return (
    <div>
      { /* رویداد ماوس */ }
      <div>
        onMouseEnter={() => console.log("ماوس وارد شد")}
        onMouseLeave={() => console.log("ماوس خارج شد")}
      >
        منطقه حساس
      </div>

      { /* رویداد کیبورد */ }
      <input
        onKeyDown={(e) => console.log("کلید فشرده شد:", e.key)}
        onFocus={() => console.log("فوکوس گرفت")}
        onBlur={() => console.log("فوکوس از دست رفت")}
      />
    </div>
  );
}

```

Conditional Rendering (رندر شرطی)

روش ۱: عملگر `&&` (AND منطقی)

```
import React from "react";

function ConditionalDisplay() {
  const isLoggedIn = true;
  const hasMessages = false;

  return (
    <div>
      {/* بودن true نمایش در صورت */}
      {isLoggedIn && <p>خوش آمدید کاربر</p>}

      {/* بودن false نمایش در صورت */}
      {hasMessages && <p>پیام جدید دارید</p>}

      {/* ترکیب چند شرط */}
      {isLoggedIn && !hasMessages && (
        <p>شما پیام جدیدی ندارید</p>
      )}
    </div>
  );
}

export default ConditionalDisplay;
```

اگر می‌خواستیم همین مثال را با *if* حل کنیم باید:

```
import React from "react";

function ConditionalDisplay() {
  const isLoggedIn = true;
  const hasMessages = false;

  // ۱. If not logged in, show a completely different UI
  if (!isLoggedIn) {
    return <p>لطفا ابتدا وارد حساب خود شوید</p>;
  }

  // ۲. If logged in but has messages
  if (hasMessages) {
    return (
      <div>
        <p>خوش آمدید</p>
      </div>
    );
  }
}
```

```

        <p>شما پیام جدید دارید.</p>
      </div>
    );
  }

  // ۳. Default case (Logged in, no messages)
  return (
    <div>
      <p>خوش آمدید</p>
      <p>شما پیام جدیدی ندارید.</p>
    </div>
  );
}
export default ConditionalDisplay;

```

روش ۲: *Ternary Operator* (?:)

```

import React from "react";

function UserStatus() {
  const isPremium = true;

  return (
    <div>
      <p>
        وضعیت حساب:
        {isPremium ? "کاربر ویژه 🏆" : "کاربر معمولی"}
      </p>

      { /* استفاده برای نمایش کامپوننت‌های مختلف */ }
      {isPremium ? (
        <div className="premium-features">
          <h۳>امکانات ویژه</h۳>
          <ul>
            <li>امکان ۱</li>
            <li>امکان ۲</li>
          </ul>
        </div>
      ) : (
        <button>ارتقا به حساب ویژه</button>
      ) }
    </div>
  );
}
export default UserStatus;

```


روش ۳: متغیر شرطی

```
function MessageComponent() {
  const isLoading = true;

  let content;

  if (isLoading) {
    content = <p>در حال بارگذاری</p>;
  } else {
    content = <p>محتوای اصلی</p>;
  }

  return <div>{content}</div>;
}
```

۶. رندر لیست‌ها و استفاده از `key`

رندر آرایه ساده

```
function SimpleList(){
  const color = ["yellow","blue","green","red"];
  return (
    <ul>
      {color.map((color,index)=>(
        <li key={index}>{color}</li>
      ))}
    </ul>
  )
}
```

رندر لیست اشیاء

```
function StudentList() {
  const students = [
    { id: ۱, name: "علی رضایی", grade: ۱۸,۵ },
    { id: ۲, name: "مریم محمدی", grade: ۱۹,۲ },
    { id: ۳, name: "حسین کریمی", grade: ۱۷,۸ },
  ];

  return (
    <table>
      <thead>
        <tr>
          <th>ردیف</th>
          <th>نام</th>
        </tr>
      </thead>
      <tbody>
        {students.map((student, index) => (
          <tr>
            <td>{index+1}</td>
            <td>{student.name}</td>
            <td>{student.grade}</td>
          </tr>
        ))}
      </tbody>
    </table>
  )
}
```

```

        <th>نمره</th>
      </tr>
    </thead>
    <tbody>
      {students.map((student, index) => (
        <tr key={student.id}>
          <td>{index + 1}</td>
          <td>{student.name}</td>
          <td>{student.grade}</td>
        </tr>
      ))}
    </tbody>
  </table>
);
}

```

توجه *key* تنها به فرزند اصلی تابع *map* داده میشود.

اهمیت *'key'*

```

function KeyImportance() {
  const items = [
    { id: "uniq-۱", value: "آیتم اول" },
    { id: "uniq-۲", value: "آیتم دوم" },
    { id: "uniq-۳", value: "آیتم سوم" },
  ];

  return (
    <div>
      <h۴>مناسب key لیست با</h۴>
      <ul>
        {items.map(item => (
          <li key={item.id}>{item.value}</li>
        ))}
      </ul>

      <h۴>key به عنوان index مشکل استفاده از</h۴>
      <ul>
        {items.map((item, index) => (
          <li key={index}>
            {item.value} (key={index})
          </li>
        ))}
      </ul>
    </div>
  );
}

```

در ری اکت، *key* یک ویژگی (*Prop*) خاص است که به این کتابخانه کمک می‌کند تا تشخیص دهد کدام آیتم‌ها در یک لیست تغییر کرده‌اند، اضافه شده‌اند یا حذف شده‌اند. بدون داشتن یک کلید یکتا، ری اکت مجبور است کل لیست را از نو رندر کند که باعث افت عملکرد و بروز باگ‌های عجیب می‌شود.

۱. الگوریتم *Diffing* (مقایسه)

ری‌اکت از یک "*DOM* مجازی" استفاده می‌کند. وقتی لیستی تغییر می‌کند، ری‌اکت نسخه جدید را با نسخه قبلی مقایسه می‌کند. *key* مانند یک *کد ملی* برای هر المان عمل می‌کند. ری‌اکت با چک کردن این کد، می‌فهمد که مثلاً آیتم دوم فقط جابه‌جا شده است و نیازی نیست آن را حذف و دوباره بسازد.

۲. حفظ وضعیت (*State*)

اگر از *key* استفاده نکنید یا از *index* (شماره ردیف) استفاده کنید، در صورت جابه‌جا شدن آیتم‌ها، ری‌اکت گیج می‌شود.

- مثال: فرض کنید یک لیست از اینپوت‌ها دارید که در هر کدام چیزی تایپ کرده‌اید. اگر یک آیتم به بالای لیست اضافه شود، اگر از *index* استفاده کرده باشید، نوشته‌های شما در اینپوت‌ها جابه‌جا نمی‌شوند و در همان ردیف قبلی باقی می‌مانند، چون ری‌اکت فکر می‌کند ردیف ۰ همان ردیف ۰ قبلی است!

تفاوت استفاده از *id* و *index* در مثال شما

ویژگی	استفاده از <i>item.id</i> (مناسب)	استفاده از <i>index</i> (نامناسب)
ثبات	حتی با حذف یا اضافه کردن آیتم، <i>ID</i> ثابت می‌ماند.	با حذف یک آیتم، ایندکس بقیه آیتم‌ها تغییر می‌کند.
عملکرد	ری‌اکت فقط تغییرات واقعی را اعمال می‌کند.	ری‌اکت ممکن است بخش زیادی از لیست را بیهوده رندر کند.
باگ در فرم‌ها	مقادیر داخل اینپوت‌ها به درستی حفظ می‌شوند.	ممکن است مقدار تایپ شده در ردیف ۱ به ردیف ۲ منتقل شود.

قوانین طلایی برای انتخاب *key*

۱. **یکتا بودن در سطح لیست:** کلید باید در بین همسایه‌های خودش در آن لیست منحصر به فرد باشد (نیازی نیست در کل پروژه یکتا باشد).

۲. **ثابت بودن:** کلید نباید با هر بار رندر تغییر کند (مثلاً استفاده از *Math.random* () برای کلید، فاجعه‌بار است چون در هر رندر یک کلید جدید ساخته می‌شود و کل لیست از نو ساخته می‌شود).

۳. **ترجیحاً ID پایگاه داده:** بهترین گزینه استفاده از *ID* هایی است که از سمت سرور یا دیتابیس می‌آیند.

نکته: فقط زمانی از *index* استفاده کنید که لیست شما کاملاً **استاتیک** باشد (یعنی هرگز حذف، اضافه یا مرتب‌سازی نشود).

فیلتر و مرتب‌سازی لیست‌ها

```
function FilteredList() {
  const products = [
    { id: ۱, name: "لپ‌تاپ", price: ۱۵۰۰۰۰۰۰, inStock: true },
    { id: ۲, name: "موبایل", price: ۸۰۰۰۰۰۰, inStock: false },
    { id: ۳, name: "تبلت", price: ۶۰۰۰۰۰۰, inStock: true },
    { id: ۴, name: "هدفون", price: ۱۵۰۰۰۰۰, inStock: true },
  ];

  // فیلتر محصولات موجود
  const availableProducts = products.filter(product => product.inStock);

  // مرتب‌سازی بر اساس قیمت
  const sortedProducts = [...products].sort((a, b) => a.price - b.price);

  return (
    <div>
      <h۳>محصولات موجود</h۳>
      <ul>
        {availableProducts.map(product => (
          <li key={product.id}>
            {product.name} - {product.price.toLocaleString()} تومان
          </li>
        ))}
      </ul>
    </div>
  );
}
```

۱. تعریف منبع داده (*Data Source*)

در ابتدای تابع، یک آرایه به نام *products* تعریف شده که شامل اشیاء (*Objects*) مختلف است. هر محصول دارای ویژگی‌های زیر است:

- *id*: یک شناسه یکتا (بسیار مهم برای رندر کردن لیست).
- *name* و *price*: اطلاعات نمایشی.
- *inStock*: یک مقدار *Boolean* که موجود بودن کالا را نشان می‌دهد.

۲. فیلتر کردن داده‌ها (*Filtering*)

```
const availableProducts = products.filter(product => product.inStock);
```

متد *filter*() یک آرایه جدید ایجاد می‌کند. شرط داخل آن می‌گوید: «فقط محصولاتی را نگه دار که *inStock* آن‌ها برابر با *true* باشد.» در اینجا محصولات با *ID* شماره ۱، ۳ و ۴ انتخاب می‌شوند و موبایل (*ID* ۲) حذف می‌شود.

۳. مرتب‌سازی (*Sorting*)

```
const sortedProducts = [...products].sort((a, b) => a.price - b.price);
```

در این خط، دو نکته حرفه‌ای وجود دارد:

- **کپی کردن با *Spread Operator* (*[...]*):** متد *sort* آرایه اصلی را تغییر می‌دهد (*Mutate* می‌کند). با استفاده از *[...]* ابتدا یک کپی از آرایه می‌گیریم تا آرایه اصلی *products* دست‌نخورده باقی بماند.
- **منطق مرتب‌سازی:** محصولات را از ارزان‌ترین به گران‌ترین مرتب می‌کند.

نکته: در بخش *return* کد شما، از *availableProducts* استفاده شده، یعنی محصولات فیلتر شده نمایش داده می‌شوند اما ترتیب آن‌ها بر اساس قیمت نیست (چون از *sortedProducts* استفاده نکردید).

۴. رندر کردن لیست (*Mapping*)

در بخش *JSX*، از متد *map*() برای تبدیل آرایه داده‌ها به عناصر *HTML* استفاده شده است:

- `key={product.id}`: همان‌طور که قبلاً بحث کردیم، اینجا از **ID** یکتای هر محصول استفاده شده تا ریاکت در مدیریت لیست بهینه عمل کند.

- `toLocaleString()`: یک متد عالی برای اعداد است که قیمت را به صورت سه‌رقم سه‌رقم جدا می‌کند (مثلاً ۱۵,۰۰۰,۰۰۰) تا خوانایی برای کاربر ایرانی بیشتر شود.

خروجی نهایی کد شما در مرورگر:

- لپ‌تاپ - ۱۵,۰۰۰,۰۰۰ تومان
- تبلت - ۶,۰۰۰,۰۰۰ تومان
- هدفون - ۱,۵۰۰,۰۰۰ تومان

۷. تمرین عملی ترکیبی

```
function TaskManager() {
  const tasks = [
    { id: ۱, text: "یادگیری React", completed: true },
    { id: ۲, text: "تمرین JSX", completed: false },
    { id: ۳, text: "پروژه عملی", completed: false },
  ];

  const totalTasks = tasks.length;
  const completedTasks = tasks.filter(task => task.completed).length;

  return (
    <div className="task-manager">
      <h۲>مدیریت وظایف ({completedTasks}/{totalTasks})</h۲>

      { /* Conditional Rendering برای لیست خالی */ }
      {tasks.length === ۰ ? (
        <p>هیچ وظیفه‌ای وجود ندارد</p>
      ) : (
        <ul>
          {tasks.map(task => (
            <li
              key={task.id}
              // Conditional styling
              style={{
                color: task.completed ? "green" : "black",
                textDecoration: task.completed ? "line-through" : "none"
              }}
            >
              {task.text}
              <button
```

```

        onClick={() => console.log("تغییر وضعیت:", task.id)}
      >
      {task.completed ? "تکمیل" : "بازنشانی"}
    </button>
  </li>
)}
</ul>
)}
</div>
);
}

```

خلاصه قوانین JSX

قانون	مثال صحیح	مثال غلط
تگ بسته	<code><div></div></code> <code></code>	<code><div></code> <code>
</code>
یک العنت والد	<code><div><p>۱</p><p>۲</p></div></code>	<code><p>۱</p><p>۲</p></code>
<i>Expression</i> در <code>{}</code>	<code><p>{name}</p></code>	<code><p>"name"</p></code>
رویدادها	<code>onClick={handleClick}</code>	<code>onClick="handleClick()"</code>
<i>key</i> در لیست‌ها	<code>key={item.id}</code>	<code>key={index}</code>

نکات کلیدی برای یادگیری

۱. *JSX* در نهایت به کد جاوااسکریپت معمولی تبدیل می‌شود
۲. هرچه در *JSX* می‌نویسید باید یک *Expression* معتبر باشد
۳. `key` باید منحصر به فرد و ثابت باشد (از *index* فقط در موارد خاص استفاده کنید)
۴. رویدادهای *React* با رویدادهای *HTML* تفاوت دارند (*camelCase*)
۵. *Fragment* (`</>...</>`) برای گروه‌بندی العنت‌ها بدون اضافه کردن *DOM* اضافی

فصل سوم: مدیریت *State* و *Props* (۲ جلسه)

هدف: دانشجو بتواند داده را بین کامپوننت‌ها منتقل کند و *state* کامپوننت خود را مدیریت نماید.

۱. آشنایی با *Props*

- * ارسال داده از *Parent* به *Child*
- * *Type Checking* با *PropTypes* یا *PropTypes* (به صورت مقدماتی)

آموزش گام به گام: ایجاد کامپوننت `ProductList`

۲. مدیریت *State* با هوک `useState`

- * معرفی هوک‌ها (*Hooks*) و قوانین آن‌ها
- * استفاده از `useState` برای مدیریت *State* در کامپوننت‌های تابعی
- * تفاوت بین *State* و *Props*
- * آپدیت *State* بر اساس مقدار قبلی (با استفاده از *Callback Function*)

مفهوم *Props*: ارسال داده از *Parent* به *Child*

در ری‌اکت، جریان داده‌ها یک‌طرفه است (از بالا به پایین). یعنی کامپوننت والد (*Parent*) می‌تواند اطلاعاتی را به کامپوننت فرزند (*Child*) بفرستد، اما فرزند مستقیماً نمی‌تواند *Props* را تغییر دهد (*Props* در فرزند *Read-only* هستند).

مثال عملی:

فرض کنید یک پوشه به نام *components* دارید.

فایل فرزند: `src/components/UserCard.jsx`

یا در تایپ اسکریپت `src/components/UserCard.tsx`

این کامپوننت فقط وظیفه نمایش اطلاعاتی را دارد که به او "پاس داده می‌شود".

```
function UserCard(props) {
  return (
    <div style={{ border: '1px solid #ccc', padding: '10px', margin: '10px' }}>
      <h2>نام: {props.name}</h2>
      <p>شغل: {props.job}</p>
      <p>وضعیت: {props.isOnline ? "🟢 آفلاین" : "🔴 آنلاین"}</p>
    </div>
  );
};
```



```
}  
  
export default UserCard;
```

فایل والد: *App.jsx*

یا در تایپ اسکریپت *App.tsx*

در اینجا ما از *UserCard* استفاده می‌کنیم و داده‌های مختلف را برای هر نمونه ارسال می‌کنیم.

```
import UserCard from '../components/UserCard';  
  
function App() {  
  return (  
    <div>  
      <h1>لیست اساتید دانشکده</h1>  
      { /* ارسال داده‌ها از طریق ویژگی‌های دلخواه */ }  
      <UserCard name="دکتر رضایی" job="استاد هوش مصنوعی" isOnline={true} />  
      <UserCard name="مهندس علوی" job="مدرس پایتون" isOnline={false} />  
    </div>  
  );  
}  
  
export default App;
```

[vite-project\src\App.jsx](#)

مثال دوم

۱. فایل والد: *src/App.jsx*

```
import ProductList from '../components/ProductList';  
  
function App() {  
  // ۱. تعریف منبع داده (داده‌ها در سطح بالا مدیریت می‌شوند)  
  const products = [  
    { id: ۱, name: "لپ‌تاپ ایسوس", price: ۴۵۰۰۰۰۰ },  
    { id: ۲, name: "موس گیمینگ", price: ۱۲۰۰۰۰۰ },  
    { id: ۳, name: "کیبورد مکانیکی", price: ۳۵۰۰۰۰۰ },  
  ];  
}
```

```
// در والد (Logic) تعریف تابع عملیاتی ۲.
const handleAction = (name) => {
  console.log("کلیک شد " + name + " پیام از والد: دکمه مربوط به");
};

return (
  <div style={{ direction: 'rtl', padding: '۲۰px' }}>
    <h۱>فروشگاه دانشکده</h۱>

    { /* ۳. ارسال لیست و تابع از طریق Props */ }
    <ProductList items={products} onBtnClick={handleAction} />
  </div>
);
}

export default App;
```

توضیحات فایل *App* (والد):

- مدیریت داده: طبق اصول ری‌اکت، "منبع حقیقت" (*Source of Truth*) یا همان آرایه محصولات را در والد قرار دادیم تا کنترل متمرکز داشته باشیم.
- تعریف رفتار (*Behavior*): تابعی به نام *handleAction* ساخته‌ایم. دقت کنید که این تابع در اینجا اجرا نمی‌شود، بلکه فقط نام آن به عنوان یک "بسته" به فرزند فرستاده می‌شود.
- ارسال *Props*: ما دو *Prop* تعریف کردیم؛ یکی *items* برای انتقال داده و دیگری *onBtnClick* برای انتقال رفتار.

۲. فایل فرزند: *src/components/ProductList.jsx*

یا در تایپ اسکریپت *src/components/ProductList.tsx*

```
function ProductList({ items, onBtnClick }) {
  return (
    <div style={{ border: '۱px solid #ddd', padding: '۱۰px' }}>
      <h۳>لیست محصولات موجود</h۳>
      <ul>
        {items.map((product) => (
          <li key={product.id} style={{ marginBottom: '۱۰px' }}>
            <span>{product.name}</span>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```

        { /* دکمه‌ای که تابع والد را فراخوانی می‌کند */
        <button
          onClick={() => onBtnClick(product.name)}
          style={{ marginRight: '10px' }}
        >
          نمایش در کنسول
        </button>
      </li>
    )}
  </ul>
</div>
);
}

export default ProductList;

```

توضیحات فایل *ProductList* (فرزند):

- دریافت *Props*: با استفاده از تکنیک *Destructuring*، مستقیم به *items* و *onBtnClick* دسترسی پیدا کردیم.
- نمایش لیست: با متد *map()* روی آرایه دریافتی پیمایش کردیم تا به ازای هر محصول یک ردیف بسازیم.
- ارتباط معکوس (*Callback*): در قسمت *onClick* دکمه، ما تابع *onBtnClick* را که از والد گرفته بودیم، "صدا می‌زنیم". با این کار، وقتی کاربر روی دکمه در فرزند کلیک می‌کند، در واقع کدی که در والد نوشته شده بود اجرا می‌شود.

خلاصه کلیدی:

این ساختار نشان می‌دهد که فرزند فقط یک نمایش‌دهنده (*Presenter*) است و تمام تصمیمات مهم (داده‌ها و لاگ گرفتن) در والد اتخاذ می‌شود. این کار باعث می‌شود کامپوننت *ProductList* قابل استفاده مجدد (*Reusable*) باشد، چون می‌توان آن را با لیست‌های دیگر و توابع متفاوت در جاهای دیگر پروژه هم به کار برد.

۲. بررسی نوع داده (*Type Checking*)

وقتی در یک پروژه تیمی کار می‌کنید، ممکن است همکار شما فراموش کند که **age** باید عدد باشد یا **name** الزامی است. برای جلوگیری از باگ‌های احتمالی، از **PropTypes** استفاده می‌کنیم.

استفاده از *PropTypes* (مقدماتی)

ابتدا باید آن را نصب یا وارد کنید: `import PropTypes from 'prop-types'`;

در دنیای ری‌اکت، **PropTypes** ابزاری برای تأیید نوع (*Type Validation*) داده‌های ورودی یک کامپوننت است. به زبان ساده، پروپ-تایپس مانند یک نگهبان عمل می‌کند که چک می‌کند آیا داده‌هایی که از والد به فرزند پاس داده شده‌اند، از نوع صحیح (رشته، عدد، شیء و ...) هستند یا خیر.

اگر نوع داده اشتباه باشد، ری‌اکت در کنسول مرورگر به شما اخطار (*Warning*) می‌دهد. این کار به خطایابی سریع‌تر در پروژه‌های بزرگ کمک می‌کند.

۱. نحوه نصب (*Installation*)

در نسخه‌های قدیمی ری‌اکت، این قابلیت در خود پکیج ری‌اکت بود، اما الآن باید آن را جداگانه نصب کنید. در ترمینال پروژه خود دستور زیر را وارد کنید:

با استفاده از *npm*:

```
npm install prop-types
```

۲. نحوه استفاده در کد

فرض کنید یک کامپوننت دارید که مشخصات کاربر را نمایش می‌دهد. می‌خواهید مطمئن شوید که نام حتماً *string* و سن حتماً *number* باشد.

مثال عملی:

```
import PropTypes from 'prop-types'; // ۱. وارد کردن کتابخانه

function UserProfile({ name, age, isAdmin }) {
  return (
    <div>
      <h۱>نام: {name}</h۱>
      <p>سن: {age}</p>
```

```

        <p>وضعیت مدیریت: {isAdmin ? "خیر": "بله"}</p>
    </div>
    );
}

// ۲. تعریف قوانین برای پروپ‌ها
UserProfile.propTypes = {
    name: PropTypes.string.isRequired, // حتماً باید رشته باشد و اجباری است
    age: PropTypes.number,              // باید عدد باشد
    isAdmin: PropTypes.bool             // باید بولین باشد
};

// ۳. تعیین مقادیر پیش‌فرض (اختیاری)
UserProfile.defaultProps = {
    age: ۲۰,
    isAdmin: false
};

export default UserProfile;

```

۳. چرا از *TypeScript* استفاده کنیم؟ (نگاهی گذرا)

در پروژه‌های مدرن دانشگاهی و صنعتی، به جای *PropTypes* از *TypeScript* استفاده می‌شود چون خطاها را "هنگام کدنویسی" (قبل از اجرا) به شما نشان می‌دهد.

مثال با *TypeScript*:

```

interface UserProps {
    name: string;
    job: string;
    isOnline: boolean;
}

const UserCard: React.FC<UserProps> = ({ name, job, isOnline }) => {
    return (
        <div>{name} - {job}</div>
    );
};

```

خلاصه برای مرور سریع:

۱. *Props* برای انتقال داده از والد به فرزند استفاده می‌شود.

۲. **ReadOnly**: کامپوننت فرزند نباید **props** را تغییر دهد.

۳. **Destructuring**: به جای نوشتن **props.name** بهتر است از **{ name }** در ورودی تابع استفاده کنیم.

۴. **Validation**: با **PropTypes** یا **TS** مطمئن می‌شویم که داده‌های ورودی سالم و صحیح هستند.

تمرین برای شما: سعی کنید یک کامپوننت به نام **Product** بسازید که **price** و **discount** را به عنوان **Props** بگیرد و قیمت نهایی را محاسبه و چاپ کند. آیا می‌خواهید در انجام این تمرین به شما کمک کنم؟

آموزش گام به گام: ایجاد کامپوننت ``ProductList``

گام ۱: تعریف سناریو (کامپوننت والد: ``App.jsx``)

هدف ما ساخت یک صفحه (کامپوننت ``App``) است که لیست محصولات را نمایش می‌دهد. کامپوننت والد مسئول نگهداری داده‌ها و تعریف منطق عملیاتی است.

 فایل ``App.jsx`` (وضعیت اولیه)

```
// App.jsx

import React from 'react';
import './App.css';

// [فراخوانی شود ProductList اینجا قرار است کامپوننت]

function App() {
  // وظیفه نگهداری و مدیریت داده‌ها را دارد App (Data): داده‌ها ۱.
  const sampleProducts = [
    { id: ۱, name: "لپ‌تاپ ایسوس", price: ۲۰۰۰۰۰۰ },
    { id: ۲, name: "ماوس بی‌سیم", price: ۵۰۰۰۰ },
    { id: ۳, name: "مانیتور ۲۴ اینچ", price: ۸۰۰۰۰۰ }
  ];
}
```

```
// تعریف تابعی که قرار است در کامپوننت فرزند اجرا شود: منطق عملیاتی ۲.
const handleAddToCart = (product) => {
  alert(`به سبد خرید اضافه شد "${product.name}" محصول`);
};

return (
  <div className="App">
    <h1>🛒 فروشگاه کوچک من</h1>

    { /* اینجا قرار خواهیم داد ProductList در گام ۳، ما */ }
    { /* <ProductList products={sampleProducts}
onAddToCart={handleAddToCart} /> */ }

  </div>
);
}

export default App;
```

لینک به فایل [src\App.jsx](#)

توضیح گام به گام کامپوننت *App.jsx*

کامپوننت *App* در اینجا به عنوان کامپوننت والد (*Parent Component*) عمل می‌کند و هسته برنامه را تشکیل می‌دهد. وظیفه اصلی آن آماده‌سازی داده‌ها و توابعی است که قرار است به کامپوننت‌های فرزند (مانند *ProductList*) منتقل شوند.

وارد کردن کتابخانه‌ها و فایل‌های مورد نیاز

```
import React from 'react';
import './App.css';
```

- `import React from 'react'`: این خط کتابخانه اصلی *React* را وارد می‌کند. اگرچه در نسخه‌های جدیدتر *React* ممکن است نیاز به وارد کردن صریح آن نباشد، اما برای اطمینان از سازگاری و وضوح کد همچنان استفاده می‌شود. (یعنی ممکنه اگه حذفش کنی هم کار کنه ولی میذار باشه.)
- `import './App.css'`: این خط فایل استایل مربوط به این کامپوننت را وارد می‌کند تا ظاهر بصری (مانند کلاس *App*) تعریف شود.

تعریف کامپوننت *App* (تابع اصلی)

```
function App() {  
  // محتوای کامپوننت ...  
  return (  
    // ... JSX کامپوننت  
  );  
}
```

- اینجا یک کامپوننت تابعی (*Functional Component*) به نام *App* تعریف می‌شود. نام کامپوننت‌ها در *React* معمولاً با حرف بزرگ شروع می‌شود.

تعریف داده‌های برنامه (*Data*)

```
// ۱. داده‌ها (Data): App دارای مدیریت داده‌ها را دارد  
const sampleProducts = [  
  { id: ۱, name: "لپ‌تاپ ایسوس", price: ۲۰۰۰۰۰۰ },  
  { id: ۲, name: "ماوس بی‌سیم", price: ۵۰۰۰۰ },  
  { id: ۳, name: "مانیتور ۲۴ اینچ", price: ۸۰۰۰۰۰ }  
];
```

- نقش والد: این آرایه (*sampleProducts*) شامل داده‌های خام محصولات است. در معماری *React*، کامپوننت والد (*App*) مسئول نگهداری این نوع داده‌ها است و آن‌ها را به کامپوننت‌های فرزندی که نیاز به نمایش آن‌ها دارند، پاس می‌دهد.
- فعلاً این داده‌ها ثابت هستند (نه *State*)، اما اگر قرار بود محصول جدیدی اضافه یا حذف شود، از *State* برای مدیریت آن‌ها استفاده می‌شد.

تعریف منطق عملیاتی (*Logic*)

```
// ۲. منطق عملیاتی (Logic):  
const handleAddToCart = (product) => {  
  alert(`به سبد خرید اضافه شد "${product.name}" محصول`);  
};
```

- تابع *Callback*: این تابع (*handleAddToCart*) در کامپوننت والد تعریف شده است.
- هدف این است که این تابع به عنوان یک *Prop* به کامپوننت فرزند (*ProductList*) ارسال شود.

- وقتی کاربر در کامپوننت فرزند روی دکمه‌ای کلیک می‌کند، آن کامپوننت این تابع را فراخوانی می‌کند (*Callback*). این مثال اصلی جریان داده یک‌طرفه (*Action Up*) است.

بازگرداندن رابط کاربری (*JSX*)

```
return (
  <div className="App">
    <h1>🛒 فروشگاه کوچک من </h1>

    { /* اینجا قرار خواهیم داد ProductList در گام ۳، ما */ }
    { /* <ProductList products={sampleProducts} onAddToCart={handleAddToCart} */ }
  </div>
);
```

- *JSX*: این ساختار شبیه *HTML* است و نمای ظاهری کامپوننت را تعریف می‌کند. کامپوننت‌ها باید یک المان والد تکی را بازگردانند (اینجا `<div className="App">`).

- فراخوانی فرزند (خط کامنت شده): خط کامنت شده نشان‌دهنده محل و نحوه فراخوانی کامپوننت *ProductList* (که در فایل موجود نیست) است.

○ `</... ProductList>`: کامپوننت فرزند فراخوانی شده است.

○ `products={sampleProducts}`: داده‌ها (*Props*) به فرزند پاس داده می‌شوند (*Data Down*).

○ `onAddToCart={handleAddToCart}`: تابع *Callback* (منطق عملیاتی) به فرزند پاس داده می‌شود (*Action Up*).

صادرات کامپوننت (*Export*)

```
export default App;
```

- این خط باعث می‌شود که کامپوننت *App* قابل دسترسی و استفاده در فایل‌های دیگر (مانند *index.js* یا *main.js*) باشد تا بتواند برنامه شما را در مرورگر رندر کند.

جمع‌بندی: این کد، نمونه‌ای کامل از نحوه تفکیک وظایف است:

- **App**: داده‌ها و توابع را مدیریت می‌کند (داده‌ها به پایین، توابع به بالا).
- **ProductList** (فرض شده): وظیفه صرفاً نمایش و فراخوانی توابع والد را دارد.

گام ۲: ایجاد و تعریف کامپوننت فرزند (*ProductList*)

حالا یک کامپوننت تابعی به نام *ProductList* می‌سازیم که وظیفه‌اش فقط نمایش داده‌هایی است که از والد دریافت می‌کند.

 تعریف کامپوننت *ProductList* در همان فایل *App.jsx* (برای سادگی)

```
// کامپوننت ProductList
function ProductList({ products, onAddToCart }) {
  // را به صورت آرگومان دریافت می‌کند Props کامپوننت فرزند

  return (
    <div className="product-list-container">
      <h۳>لیست محصولات موجود</h۳>

      {products.map(product => (
        <div key={product.id} className="product-card">
          <h۴>{product.name}</h۴>
          <p>تومان {product.price.toLocaleString()} قیمت</p>

          <button
            // ۳. onAddToCart فراخوانی تابع والد: تابع
            onClick={() => onAddToCart(product)}
          >
            افزودن
          </button>
        </div>
      ))}
    </div>
  );
}
```

توضیح گام به گام کامپوننت *ProductList*

تعریف کامپوننت تابعی و دریافت *Props*

```
function ProductList({ products, onAddToCart }) {
```

```
// به صورت آرگومان دریافت می‌کند Props کامپوننت فرزند  
// ...  
}
```

* تعریف: ما یک کامپوننت تابعی (*Functional Component*) به نام `ProductList` تعریف می‌کنیم.

* دریافت `Props`: این کامپوننت دو مقدار را به صورت `Props` (از کامپوننت والد، مانند `App`) دریافت می‌کند:

* `products`: آرایه‌ای از اشیاء محصول که باید نمایش داده شوند (داده‌های ورودی).

* `onAddToCart`: یک تابع (*Callback Function*) که والد برای مدیریت رویداد "افزودن به سبد خرید" تعریف کرده است.

`Props`ها فقط خواندنی هستند: این کامپوننت نمی‌تواند آرایه `products` یا تابع `onAddToCart` را تغییر دهد؛ فقط می‌تواند آن‌ها را استفاده کند.

بازگرداندن ساختار ظاهری (*JSX*)

```
return (  
  <div className="product-list-container">  
    <h3>لیست محصولات موجود</h3>  
    // ... لیست  
  </div>  
) ;
```

* کامپوننت *JSX* مورد نیاز برای نمایش بخش لیست محصولات را باز می‌گرداند. این ساختار شامل یک عنوان ساده (`<h3>`) و یک کانتینر اصلی (`<div className="product-list-container">`) است.

پیمایش و رندر مشروط لیست محصولات (*Mapping*)

```
{products.map(product => (  
  <div key={product.id} className="product-card">  
    // ... جزئیات محصول  
  </div>  
))}
```

پیمایش: با استفاده از متد `map` جاوااسکریپت، روی آرایه `products` (که به عنوان *Prop* دریافت شده) پیمایش می‌شود.

تولید *JSX*: به ازای هر شیء `product` در آرایه، یک بلاک *JSX* (کارت محصول) ایجاد و رندر می‌شود.

`key` ضروری: ویژگی `key={product.id}` به *React* کمک می‌کند تا هر آیتم لیست را به طور یکتا شناسایی کرده و عملکرد خود را در هنگام به‌روزرسانی لیست‌ها بهینه‌سازی کند.

نمایش جزئیات محصول

```
<h4>{product.name}</h4>
<p>تومان {product.price.toLocaleString()} : قیمت</p>
```

نمایش داده‌ها: مقادیر موجود در شیء `product` (مانند `name` و `price`) مستقیماً درون *JSX* با استفاده از بریس‌ها (`{}`) نمایش داده می‌شوند. این بخش نمایش مستقیم جریان داده از بالا به پایین است.

گام ۵: مدیریت رویداد کاربر و اطلاع‌رسانی به والد

```
<button
  // ۳. فراخوانی تابع والد: تابع
  onClick={() => onAddToCart(product)}
>
  افزودن
</button>
```

* رویداد `onClick`: یک *Event Listener* به دکمه اضافه شده است.

* فراخوانی *Callback*: هنگامی که کاربر روی دکمه کلیک می‌کند، تابع `onAddToCart` (که به عنوان *Prop* دریافت شده) فراخوانی می‌شود.

* ارسال داده: در زمان فراخوانی، شیء `product` فعلی به عنوان آرگومان به تابع ارسال می‌شود.

* نتیجه: این عمل، جریان داده از پایین به بالا (*Action Up*) را فعال می‌کند، به این معنی که کامپوننت فرزند، رویداد و داده‌های مربوطه را به تابع تعریف شده در کامپوننت والد برمی‌گرداند تا والد بتواند منطق مورد نظر (مثل به‌روزرسانی سبد خرید) را اجرا کند.

گام ۳: فراخوانی کامپوننت فرزند و ارسال داده‌ها (*Props*)

در نهایت، کامپوننت `App` باید کامپوننت `ProductList` را در بخش `return` خود استفاده کند و داده‌ها و توابع خود را به آن پاس دهد.

📄 به روزرسانی بخش `return` در کامپوننت `App`

```
// ... در کامپوننت App

return (
  <div className="App">
    <h1>🛒 فروشگاه کوچک من </h1>

    { /* فراخوانی و ارسال Props */ }
    <ProductList
      products={sampleProducts} // ارسال داده‌ها
      onAddToCart={handleAddToCart} // ارسال تابع
    />

  </div>
);
// ...
```

با این کار، جریان داده بین دو کامپوننت برقرار می‌شود:

۱. داده‌ها از `App` به `ProductList` پایین می‌آیند.
۲. عملیات‌ها (کلیک کاربر) از `ProductList` به `App` بازگشت داده می‌شوند (از طریق فراخوانی تابع `onAddToCart`).

مفهوم جریان داده یک‌طرفه (*Unidirectional Data Flow*)

مهم‌ترین اصل در معماری کامپوننت‌ها در **React**، جریان داده یک‌طرفه (**One-Way Data Flow**) است. این یعنی:

۱. داده‌ها همیشه از بالا به پایین حرکت می‌کنند. یعنی از کامپوننت والد (مانند `App`) به کامپوننت‌های فرزند (مانند `ProductList`) از طریق **Props** منتقل می‌شوند.
۲. کامپوننت فرزند نمی‌تواند مستقیماً **Prop** دریافتی خود را تغییر دهد. **Props**‌ها برای کامپوننت‌های فرزند فقط خواندنی (**Read-Only**) هستند.

مثال عملیاتی در کد:

* جریان پایین (**Data Down**):

* کامپوننت ``App`` متغیر ``sampleProducts`` را به عنوان `Prop` به نام ``products`` به ``ProductList`` می‌دهد.

* ``ProductList`` این لیست را دریافت کرده و آن را نمایش می‌دهد.

* جریان بالا (`Action Up`):

* اگر کاربر بخواهد محصولی را اضافه کند، ``ProductList`` نمی‌تواند مستقیماً آرایه محصولات در ``App`` را تغییر دهد.

* در عوض، ``App`` یک تابع `Callback` (به نام ``handleAddToCart``) را به عنوان `Prop` به ``ProductList`` می‌دهد.

* وقتی کاربر کلیک می‌کند، ``ProductList`` این تابع `Callback` را فراخوانی کرده و رویداد را به والد اطلاع می‌دهد.

* این تابع در والد (``App``) اجرا می‌شود و می‌تواند در صورت لزوم، `State` (داده‌های متغیر) والد را به‌روزرسانی کند.

این ساختار تضمین می‌کند که منطق و داده‌ها در یک مکان متمرکز هستند، که اشکال‌زدایی (`Debugging`) و پیگیری تغییرات در برنامه را بسیار آسان‌تر می‌کند.

این رویکرد، بهترین تمرین (`Best Practice`) در توسعه `React` برای سازماندهی پروژه و بهبود قابلیت نگهداری کد است. جداسازی هر کامپوننت در یک فایل مجزا، به خوانایی و ماژولار بودن پروژه کمک شایانی می‌کند.

در اینجا، من نحوه جداسازی کامپوننت‌های `App` و `ProductList` را به صورت گام به گام و آموزشی توضیح می‌دهم.

جداسازی کامپوننت‌ها در فایل‌های مجزا

تا کنون، تمام منطق و ظاهر برنامه در فایل `App.jsx` قرار داشت. برای توسعه‌پذیری بهتر، ما کامپوننت `ProductList` را به فایل مخصوص خودش منتقل می‌کنیم.

گام ۱: ایجاد ساختار پوشه

بهتر است کامپوننت‌های خود را درون یک پوشه به نام *components* یا *src/components* نگهداری کنیم.

```
src/
├── components/           پوشه جدید برای کامپوننت‌های قابل استفاده مجدد
│   └── ProductList.jsx   کامپوننت فرزند جدا شده
├── App.jsx               کامپوننت اصلی (والد)
└── index.css / App.css   فایل‌های استایل
```

ایجاد و تعریف *ProductList.jsx*

تمام کد مربوط به کامپوننت *ProductList* را از *App.jsx* جدا کرده و در فایل جدید *ProductList.jsx* قرار می‌دهیم. نکته مهم، صادرات (*Export*) این کامپوننت است تا فایل‌های دیگر بتوانند از آن استفاده کنند.

فایل *ProductList.jsx*

```
// ProductList.jsx - مسئول نمایش لیست محصولات

import React from 'react';
// اصلی نیست، مگر اینکه استایل‌های مخصوص به خود را داشته باشد CSS نیازی به وارد کردن

// کامپوننت ProductList
function ProductList({ products, onAddToCart }) {
  // تابع (onAddToCart) و تابع (products) لیست داده‌ها Prop: دریافت دو

  const totalProducts = products.length;

  return (
    <div className="product-list">
      <h۲>محصول ({totalProducts}) لیست محصولات</h۲>

      { /* منطق شرطی برای نمایش لیست یا پیام خالی */ }
      {products.length === ۰ ? (
        <p className="empty-message">محصولی یافت نشد</p>
      ) : (
        <div className="products">
          {products.map(product => (
            <div key={product.id} className="product-card">
              <h۳>{product.name}</h۳>
              <p>تومان {product.price.toLocaleString()} قیمت</p>
              <button
```

```

// دریافتی را فراخوانی کرده و محصول را به والد Prop هنگام کلیک، تابع
دریافتی را فراخوانی کرده و محصول را به والد Prop هنگام کلیک، تابع
برمی‌گرداند.

onClick={() => onAddToCart(product)}
className="add-button"
>
    افزودن به سبد خرید
</button>
</div>
    )}
  </div>
  )}
</div>
);
}

export default ProductList; // کامپوننت را صادر کنید.

```

گام ۳: به‌روزرسانی فایل والد (*App.jsx*)

در فایل *App.jsx*، باید کد کامپوننت *ProductList* را حذف کرده و در عوض، آن را با استفاده از دستور *import* وارد کنیم.

فایل نهایی *App.jsx*

```

// App.jsx - مسئول نگهداری داده‌ها و منطق اصلی

import React from 'react';
import './App.css';
// وارد کردن کامپوننت فرزند از مسیر جدید ۱.
import ProductList from './components/ProductList';

function App() {
  // داده‌های نمونه برای محصولات (Data Source)
  const sampleProducts = [
    { id: ۱, name: "لپ‌تاپ ایسوس", price: ۲۵۰۰۰۰۰۰ },
    { id: ۲, name: "ماوس بی‌سیم", price: ۵۰۰۰۰۰ },
    { id: ۳, name: "کیبورد مکانیکی", price: ۱۲۰۰۰۰۰ },
    { id: ۴, name: "مانیتور ۲۴ اینچ", price: ۸۰۰۰۰۰۰ }
  ];
}

```



```
// تابع مدیریت افزودن به سبد خرید (Logic Source)
const handleAddToCart = (product) => {
  alert(`به سبد خرید اضافه شد "${product.name}" محصول`);
  console.log("محصول اضافه شده:", product);
};

return (
  <>
    <div className="App">
      <h1>Hello World!</h1>
      <p>من است React این اولین برنامه</p>
      <div className="card">
        <button onClick={() => alert('React! سلام')}>
          کلیک کن
        </button>
      </div>

      { /* (جریان داده یک طرفه) Props استفاده از کامپوننت وارد شده و ارسال ۲۰ */ }
      <ProductList
        products={sampleProducts}
        onAddToCart={handleAddToCart}
      />
    </div>
  </>
)
}

export default App
```

مزایای جداسازی کامپوننت‌ها

جداسازی کامپوننت‌ها به شما کمک می‌کند تا به اصول توسعه مدرن نزدیک شوید:

۱. **ماژولار بودن (Modularity):** هر فایل فقط شامل کد یک کامپوننت است، که خوانایی آن را افزایش می‌دهد.
۲. **قابلیت نگهداری (Maintainability):** اگر مشکلی در نمایش لیست محصولات پیش بیاید، می‌دانید که باید فقط فایل **ProductList.jsx** را بررسی کنید.
۳. **قابلیت استفاده مجدد (Reusability):** اگر بخواهید در آینده لیست محصولات را در صفحه دیگری نمایش دهید، کافی است فقط **ProductList.jsx** را وارد و استفاده کنید.

State در ریکت چیست و چه کاربردی دارد:

State (حالت) در **React** یک شیء جاوااسکریپت داخلی است که به یک کامپوننت اجازه می‌دهد داده‌های خصوصی خود را نگهداری و مدیریت کند. این داده‌ها ماهیت پویا (**Dynamic**) و متغیر دارند و می‌توانند در طول زمان تغییر کنند.

وقتی داده‌های **State** یک کامپوننت تغییر می‌کند، **React** به صورت خودکار آن کامپوننت و تمام فرزندان وابسته به آن را دوباره رندر (**Re-render**) می‌کند تا رابط کاربری (**UI**) با داده‌های جدید مطابقت داشته باشد.

تعریف فنی

ذخیره‌ساز داده‌های پویا: **State** برای نگهداری داده‌هایی استفاده می‌شود که برای عملکرد کامپوننت ضروری هستند و احتمال تغییر آن‌ها در پاسخ به تعاملات کاربر، پاسخ‌های شبکه، یا سایر رویدادها وجود دارد.

عامل رندر مجدد: **State** قلب تعامل‌پذیری در **React** است. تغییر **State** تنها راه مجاز برای به‌روزرسانی **UI** کامپوننت بدون دستکاری مستقیم **DOM** است.

کاربرد و نحوه استفاده از State

در کامپوننت‌های تابعی مدرن، **State** توسط **Hook** اصلی **React** یعنی **useState** مدیریت می‌شود.

تعریف State با `useState`

useState یک آرایه دو عضوی را برمی‌گرداند که شامل:

۱. مقدار فعلی **State** (مثلاً **count**).

۲. تابع به‌روزرسانی آن **State** (مثلاً **setCount**).

مثال: شمارنده ساده

src/	
├ components/	پوشه جدید برای کامپوننت‌های قابل استفاده مجدد
└ ProductList.jsx	کامپوننت فرزند جدا شده

| └─ counter.jsx 👉 کامپوننت شمارنده
└─ App.jsx 👉 کامپوننت اصلی (والد)

کدهای فایل **counter.jsx**:

```
import React, { useState } from 'react';

function Counter() {
  // مقدار اولیه ۰ است: State تعریف
  const [count, setCount] = useState(0);

  // تعریف تابع بهروزرسانی
  const handleIncrement = () => {
    // استفاده از تابع count تنها راه مجاز برای تغییر است.
    setCount(count + 1);
  };

  return (
    <div>
      <p>تعداد: {count}</p>
      <button onClick={handleIncrement}>افزایش</button>
    </div>
  );
}

export default Counter
```

<vite-project/src/components/counter.jsx>

فراخوانی در فایل **App.jsx**

```
// App.jsx - مسئول نگهداری داده‌ها و منطق اصلی

import React from 'react';
import './App.css';

// وارد کردن کامپوننت فرزند از مسیر جدید ۱.
import ProductList from './components/ProductList';
import Counter from './components/counter';

function App() {
  // داده‌های نمونه برای محصولات (Data Source)
  const sampleProducts = [
    { id: ۱, name: "لپ‌تاپ ایسوس", price: ۲۵۰۰۰۰۰۰ },
    { id: ۲, name: "ماوس بی‌سیم", price: ۵۰۰۰۰۰ },
    { id: ۳, name: "کیبورد مکانیکی", price: ۱۲۰۰۰۰۰ },
  ];
}
```

```

    { id: ۴, name: "مانیتور ۲۴ اینچ", price: ۸۰۰۰۰۰۰ }
  ];

  // تابع مدیریت افزودن به سبد خرید (Logic Source)
  const handleAddToCart = (product) => {
    alert(`به سبد خرید اضافه شد "${product.name}" محصول`);
    console.log("محصول اضافه شده:", product);
  };

  return (
    <>
    <div className="App">
      <h۱>Hello World!</h۱>
      <p>من است React این اولین برنامه</p>
      <div className="card">
        <button onClick={() => alert('React سلام!')}>
          کلیک کن
        </button>
      </div>

      { /* (جریان داده یک طرفه) Props استفاده از کامپوننت وارد شده و ارسال ۲. */ }
      <ProductList
        products={sampleProducts}
        onAddToCart={handleAddToCart}
      />
      <hr />
      <Counter/>
    </div>
  </>
)
}

export default App

```

۲. اصل تغییرناپذیری (Immutability)

مهم‌ترین نکته در استفاده از **State** این است که هرگز نباید **State** را مستقیماً تغییر دهید.

غلط: ❌

```

cartItems.push(newItem); // دستکاری مستقیم آرایه State
setCartItems(cartItems); // متوجه تغییر React کنید، setState حتی اگر دوباره
                           // نمی‌شود.

```

یک مثال غلط دیگر

```
const handleIncrementWrong = () => {  
  // ❌ WRONG: Attempting to mutate the state variable directly  
  // This will NOT trigger a re-render.  
  count = count + 1;  
  
  console.log(count); // You might see the new number in the console...  
  // ...but the UI (the screen) will still show the old number!  
};
```

✓ صحیح (ایجاد کپی جدید):

برای به‌روزرسانی آرایه‌ها و اشیاء، باید یک کپی جدید از **State** ایجاد کرده و تغییرات را روی آن اعمال کنید، سپس کپی جدید را به تابع **setter** بدهید.

```
// به‌روزرسانی آرایه با استفاده از Spread Operator  
setCartItems(prevItems => [...prevItems, newItem]);
```

۳. کاربردهای رایج **State**

State در هر جایی که داده‌ها نیاز به تغییر دارند، استفاده می‌شود:

کاربرد	مثال State
ورودی‌های فرم	نگهداری متن وارد شده در یک فیلد (inputText)
وضعیت UI	وضعیت باز یا بسته بودن یک منوی کشویی (isOpen)
داده‌های اصلی	لیست آیتم‌های سبد خرید (cartItems)
تعداد و شمارش	تعداد اعلان‌های خوانده نشده (notificationCount)

State به **React** این توانایی را می‌دهد که برنامه‌های تک‌صفحه‌ای (**SPAs**) پویا و تعاملی ایجاد کند که به‌سرعت به ورودی‌های کاربر پاسخ می‌دهند.

تمرین عملی: ایجاد کامپوننت `Counter2.jsx`

از شما می‌خواهم یک کامپوننت جدید به نام `Counter2.jsx` ایجاد کنید که از هوک `useState` برای مدیریت یک مقدار عددی و همچنین تعامل با کاربر از طریق ورودی و دکمه‌ها استفاده کند.

گام ۱: ایجاد فایل کامپوننت

لطفاً یک فایل جدید با نام `Counter2.jsx` ایجاد کرده

`src/components/Counter2.jsx`

```
import React, { useState } from 'react';

function Counter() {
  // ۱. تعریف State:
  // count: مقدار فعلی شمارنده
  // setCount: تابع مجاز برای به‌روزرسانی مقدار
  const [count, setCount] = useState(0);

  // --- توابع مدیریت رویدادها ---

  // تابع افزایش: یک واحد به مقدار فعلی اضافه می‌کند.
  const handleIncrement = () => {
    // setCount را با مقدار جدید (count + ۱) می‌کنیم.
    setCount(count + ۱);
  };

  // تابع کاهش: یک واحد از مقدار فعلی کم می‌کند.
  const handleDecrement = () => {
    // از ۰ کمتر نشود (اختیاری) count مطمئن می‌شویم که
    if (count > ۰) {
      setCount(count - ۱);
    }
    // اگر نمی‌خواهید محدودیت بگذارید: setCount(count - ۱);
  };

  // تابع ریست: مقدار شمارنده را به مقدار اولیه (۰) بازمی‌گرداند.
  const handleReset = () => {
    setCount(0);
  };
}
```

```
// -----

return (
  <div className="counter-container" style={{ margin: '۲۰px', padding:
'۱۰px', border: '۱px solid #ccc', borderRadius: '۸px' }}>
    <h۳>شمارنده پیشرفته (Counter)</h۳>

    { /* نمایش مقدار فعلی State */ }

    <p style={{ fontSize: '۲۰px', fontWeight: 'bold' }}>مقدار فعلی:
{count}</p>

    { /* نمایش مقدار در یک ورودی فقط خواندنی */ }
    <input
      type="number"
      value={count}
      readOnly
      style={{ padding: '۸px', fontSize: '۱۶px', marginBottom: '۱۰px' }}
    />

    { /* دکمه‌های عملیاتی */ }
    <div style={{ display: 'flex', gap: '۱۰px' }}>
      <button
        onClick={handleDecrement}
        style={{ padding: '۱۰px ۱۰px', fontSize: '۱۸px', cursor: 'pointer'
}}
      >
        - کاهش
      </button>

      <button
        onClick={handleReset}
        style={{ padding: '۱۰px ۱۰px', fontSize: '۱۸px', cursor: 'pointer'
}}
      >
         ریست
      </button>

      <button
        onClick={handleIncrement}
        style={{ padding: '۱۰px ۱۰px', fontSize: '۱۸px', cursor: 'pointer'
}}
      >
        + افزایش
      </button>
    </div>
  </div>
);
```

```

    </div>
  </div>
);
}

export default Counter;

```

[vite-project/src/components/counter2.jsx](#)

گام ۲: تکمیل منطق توابع عملیاتی

حالا توابع زیر را در کامپوننت `Counter` تکمیل کنید:

تابع	وظیفه	نحوه به‌روزرسانی <i>State</i>
<code>handleIncrement</code>	با کلیک روی دکمه '+', یک واحد به مقدار <code>count</code> اضافه کند.	<code>setCount(count + 1)</code>
<code>handleDecrement</code>	با کلیک روی دکمه '-', یک واحد از مقدار <code>count</code> کم کند.	<code>setCount(count - 1)</code>
<code>handleReset</code>	مقدار <code>count</code> را به صفر بازگرداند.	<code>setCount(0)</code>

لطفاً کد نهایی کامپوننت `Counter2.jsx` را بنویسید که شامل تعریف این سه تابع و اتصال آن‌ها به دکمه‌های مربوطه باشد.

بعد از تکمیل این فایل، در گام بعدی آن را در `App.jsx` وارد کرده و استفاده خواهیم کرد.

آموزش مدیریت فرم و لیست کاربران در *React*

در این آموزش یاد می‌گیریم چگونه اطلاعات را از یک فرزند (*Child*) دریافت کرده و در والد (*Parent*) به یک لیست اضافه کنیم تا به صورت آنی در صفحه نمایش داده شود.

۱. درک جریان داده (*Data Flow*)

در **React**، داده‌ها معمولاً از بالا به پایین حرکت می‌کنند. اما برای اینکه یک فرم بتواند لیستی را در قطعه دیگری از برنامه به‌روزرسانی کند، باید مراحل زیر را طی کنیم:

۱. ایجاد **State** در والد: لیست کاربران باید در فایل اصلی (**App.jsx**) باشد.
۲. ارسال تابع تغییر دهنده: یک تابع به فرم می‌فرستیم تا فرم بتواند اطلاعات جدید را به والد پس بدهد.
۳. به‌روزرسانی و رندر مجدد: به محض تغییر **State**، لیست به صورت خودکار دوباره نمایش داده می‌شود.

۲. گام اول: ساختار فایل **App.jsx** (مدیریت وضعیت)

ابتدا یک وضعیت (**State**) برای ذخیره لیست کاربران تعریف می‌کنیم.

```
import React, { useState } from 'react';
import UserForm from './UserForm';

const App = () => {
  // تعریف وضعیت برای لیست کاربران (در ابتدا خالی است)
  const [userItems, setUserItems] = useState([]);

  // تابعی برای اضافه کردن کاربر جدید به لیست قبلی
  const handleAddUser = (newUser) => {
    setUserItems(prevItems => [...prevItems, newUser]);
  };

  return (
    <div className="container">
      <h1>مدیریت کاربران</h1>

      { /* به فرم Prop ارسال تابع به عنوان */ }
      <UserForm onAddUser={handleAddUser} />

      { /* نمایش لیست کاربران */ }
      <div className="user-list">
        {userItems.length === 0 ? (
          <p>هنوز کاربری ثبت نشده است</p>
        ) : (
          <ul>
            {userItems.map((user) => (
```

```

        <li key={user.id}>
            {user.name} {user.fname}
        </li>
    ))}
</ul>
    )}
</div>
</div>
    );
};

```

۳. گام دوم: ساختار فایل *UserForm.jsx* (دریافت ورودی)

در این بخش، اطلاعات را از کاربر گرفته و پس از تایید، به کامپوننت والد می‌فرستیم.

```

import React, { useState } from 'react';

function UserForm({ onAddUser }) {
    const [name, setName] = useState('');
    const [fname, setFname] = useState('');

    const handleSubmit = (e) => {
        e.preventDefault(); // جلوگیری از رفرش شدن صفحه

        if (!name || !fname) {
            alert("لطفاً تمامی فیلدها را پر کنید");
            return;
        }

        // منحصربه‌فرد ID ساخت شیء کاربر جدید با یک
        const newUser = {
            id: Date.now(),
            name,
            fname
        };

        // ارسال داده به کامپوننت والد از طریق تابع
        onAddUser(newUser);

        // خالی کردن فیلدها برای ورودی بعدی
        setName('');
        setFname('');
    };
}

```

```

return (
  <form onSubmit={handleSubmit}>
    <input
      value={name}
      onChange={(e) => setName(e.target.value)}
      placeholder="نام"
    />
    <input
      value={fname}
      onChange={(e) => setFname(e.target.value)}
      placeholder="نام خانوادگی"
    />
    <button type="submit">افزودن کاربر</button>
  </form>
);
}

```

نکات کلیدی برای یادآوری:

- **Spread Operator (...):** همیشه برای اضافه کردن به لیست در **React** از ساختار `[prevItems, ...]` `[newItem]` استفاده کنید تا به جای تغییر مستقیم متغیر، یک نسخه جدید از آرایه ساخته شود.
- **Prop Passing:** نام تابعی که در `App.jsx` می‌فرستید (مثلاً `onAddUser`) باید دقیقاً با نامی که در `UserForm` دریافت می‌کنید یکی باشد.
- **Unique Key:** در متد `map()` حتماً از یک `key` منحصر به فرد (مثل `id`) استفاده کنید تا **React** در مدیریت لیست دچار خطا نشود.

فصل چهارم: هوک‌های حیاتی و اثرات جانبی

هدف: تسلط بر مدیریت اثرات جانبی، کار با **API** ها و درک چرخه حیات کامپوننت

۱. هوک `useEffect`

- * مدیریت اثرات جانبی (**Side Effects**)
- * الگوهای `Dependency Array: []`، و بدون آرایه
- * **Cleanup Functions** (توابع پاکسازی)

۲. هوک‌های کاربردی دیگر

- * `useState` برای مدیریت **State** سراسری ساده
- * `useRef` برای دسترسی مستقیم به **DOM** و ذخیره مقادیر **Mutable**

۱. هوک *useEffect*: مدیریت اثرات جانبی

این هوک برای انجام کارهایی استفاده می‌شود که مستقیماً به رندر شدن رابط کاربری مربوط نیستند (مانند دریافت داده از سرور، تایمرها یا تغییر عنوان صفحه).

در *React*، وظیفه اصلی یک کامپوننت این است که "داده را بگیرد و *UI* (رابط کاربری) بسازد". هر چیزی که خارج از این چرخه باشد، یک اثر جانبی یا *Side Effect* است.

۱. چرا به *useEffect* نیاز داریم؟ (با مثال)

فرض کنید می‌خواهید از یک سایت دیگر (مثل سایت هواشناسی) اطلاعات بگیرید و در سایت خود نمایش دهید.

- بدون *useEffect*: اگر کد گرفتن اطلاعات را مستقیم داخل بدنه کامپوننت بنویسید، *React* با هر بار آپدیت شدن صفحه (مثلاً تغییر یک عدد)، دوباره و دوباره آن کد را اجرا می‌کند و هزاران درخواست به سرور می‌فرستد!

- با *useEffect*: شما به *React* می‌گویید: «فقط یک‌بار وقتی صفحه لود شد، برو اطلاعات هواشناسی را بگیر و دیگر با آن کاری نداشته باش.»

۲. کاربردهای رایج (کجا از آن استفاده کنیم؟)

در اینجا ۳ سناریوی پرکاربرد را بررسی می‌کنیم:

الف) فراخوانی *API* (دریافت داده)

شایع‌ترین کاربرد، گرفتن اطلاعات از دیتابیس یا اینترنت است.

```
useEffect(() => {
  fetch("https://api.example.com/users")
    .then(response => response.json())
    .then(data => setUsers(data)); // ذخیره داده در State
}, []); // آرایه خالی یعنی فقط یک‌بار هنگام باز شدن صفحه
```

ب) تعامل با دنیای خارج از *React* (*Direct DOM Manipulation*)

گاهی می‌خواهید چیزی را تغییر دهید که در کنترل مستقیم **React** نیست؛ مثلاً عنوان تب مرورگر **(Title)**.

```
useEffect(() => {
  document.title = `پیام جدید (${messages.length})`;
}, [messages]); // هر بار که لیست پیام‌ها تغییر کرد، عنوان تب عوض شود
```

ج) اشتراک‌ها (**Subscriptions**) و تایمرها

مثلاً ساخت یک ساعت دیجیتال یا گوش دادن به وضعیت آنلاین/آفلاین بودن کاربر.

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("این کد هر ثانیه اجرا می‌شود");
  }, ۱۰۰۰);

  return () => clearInterval(timer); // پاکسازی: وقتی کاربر از این صفحه رفت، تایمر
  را متوقف کن
}, []);
```

۳. تفاوت حالت‌های مختلف (به زبان ساده)

تصور کنید **useEffect** مانند یک ناظر است:

نوع تنظیمات	رفتار ناظر	مثال کاربردی
بدون آرایه	در هر پلک‌زدن (رندر) React ، این کد اجرا می‌شود.	کارهای محاسباتی بسیار خاص (کمتر استفاده می‌شود)
آرایه خالی	فقط وقتی کامپوننت متولد شد (Mount)، یک‌بار اجرا می‌شود.	دریافت لیست محصولات از سرور
با وابستگی [id]	فقط وقتی id تغییر کرد، کد را دوباره اجرا می‌کند.	نمایش جزئیات محصول جدید وقتی کاربر روی آن کلیک می‌کند

الگوهای آرایه وابستگی (**Dependency Array**):

۱. بدون آرایه: کد در هر رندر اجرا می‌شود. (توصیه نمی‌شود مگر در موارد خاص).

```
useEffect(() => { console.log("در هر رندر اجرا می‌شود"); });
```

۲. آرایه خالی []: کد فقط یکبار (زمان *Mount* شدن کامپوننت) اجرا می‌شود. مناسب برای فراخوانی *API*.

```
useEffect(() => { console.log("فقط یکبار در ابتدا"); }, []);
```

۳. همراه با وابستگی *[count]*: کد فقط زمانی اجرا می‌شود که مقدار *count* تغییر کند.

```
useEffect(() => { console.log("تغییر کرد count مقدار"); }, [count]);
```

کاربرد *useEffect* در خواندن اطلاعات

آموزش اتصال *React* به *WordPress* با استفاده از *useEffect*

در این درس یاد می‌گیریم چگونه اطلاعات یک برگه (*Page*) خاص را از سایت وردپرسی خودمان دریافت کرده و در اپلیکیشن *React* نمایش دهیم.

مرحله ۱: ساخت کامپوننت نمایش برگه

ابتدا در مسیر *src/components* یک فایل جدید به نام *WpPage.jsx* بسازید. این کامپوننت وظیفه دارد "نامک" (*Slug*) برگه را بگیرد و محتوای آن را از *API* وردپرس دریافت کند.

کد فایل *src/components/WpPage.jsx*:

```
import React, { useState, useEffect } from 'react';

const WpPage = ({ pageUrl }) => {
  const [pageData, setPageData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // ۱. از آدرس (Slug) استخراج اسلاگ
    // را جدا می‌کند "about-us" کلمه "https://site.com/about-us/" مثلاً از
    const slug = pageUrl.replace(/\/$/, "").split('/').pop();

    // ۲. وردپرس برای برگه ها API آدرس
  }, []);
};
```

```

const apiUrl = `https://katibehnaaji.com/wp-
json/wp/v2/pages?slug=${slug}`;

const fetchPage = async () => {
  setLoading(true);
  try {
    const response = await fetch(apiUrl);
    const data = await response.json();

    if (data.length > 0) {
      setPageData(data[0]); // وردپرس داده ها را به صورت آرایه میفرستد
    }
  } catch (error) {
    console.error("خطا در دریافت اطلاعات", error);
  } finally {
    setLoading(false);
  }
};

fetchPage();
}, [pageUrl]); // دوباره اجرا می شود useEffect هر زمان آدرس تغییر کند،

if (loading) return <p style={{ textAlign: 'center' }}>در حال بارگذاری</p>;

if (!pageData) return <p style={{ textAlign: 'center' }}>برگه ای پیدا
نشد.</p>;

return (
  <div style={{ direction: 'rtl', padding: '۲۰px', lineHeight: '۱.۸' }}>
    <h> dangerouslySetInnerHTML={{ __html: pageData.title.rendered }} />
    <hr />
    <div dangerouslySetInnerHTML={{ __html: pageData.content.rendered }}
  />
  </div>
);
};

export default WpPage;

```

مرحله ۲: فراخوانی در فایل اصلی (*App.jsx*)

حالا باید این کامپوننت را در فایل *App.jsx* وارد کنیم و آدرس برگه مورد نظر را به آن بدهیم.

کد فایل *src/App.jsx*:

```

import React from 'react';
import WpPage from '../components/WpPage';

function App() {
  // آدرس یکی از برگه‌های سایت وردپرسی خود را اینجا وارد کنید
  const targetPageUrl = "https://katibehnaji.com/contact-us/";

  return (
    <div className="App">
      <header style={{ backgroundColor: '#۲۸۲c۳۴', padding: '۱۰px', color: 'white', textAlign: 'center' }}>
        <h۲>آموزش اتصال به وردپرس با</h۲>
      </header>

      <main>
        <WpPage pageUrl={targetPageUrl} />
      </main>
    </div>
  );
}

export default App;

```

نکات کلیدی:

۱. چرا *useEffect*؟ چون عملیات *fetch* یک *"Side Effect"* است. ما نمی‌خواهیم رندر شدن صفحه متوقف شود؛ پس ابتدا صفحه رندر می‌شود، سپس *useEffect* اجرا شده و داده‌ها را می‌آورد.
۲. آرایه وابستگی [*pageUrl*]: به دانشجویان بگویید که اگر این آرایه را نگذارند، کد در یک حلقه بی‌نهایت می‌افتد. وجود *pageUrl* در اینجا یعنی: "فقط وقتی آدرس تغییر کرد، دوباره داده‌ها را بگیر".
۳. تفاوت *posts* و *pages*: تاکید کنید که برای نوشته‌ها از آدرس *wp/v2/posts/* و برای برگه‌ها از *wp/v2/pages/* استفاده می‌کنیم.
۴. تزریق *HTML*: وردپرس متن‌ها را به صورت کد *HTML* می‌فرستد. در *React* برای نمایش این کدها از ویژگی *dangerouslySetInnerHTML* استفاده می‌کنیم.

بکارگیری *useEffect* در تعامل با دنیای خارج از *React* (*DOM Manipulation*)

گاهی می‌خواهید چیزی را تغییر دهید که در کنترل مستقیم *React* نیست؛ مثلاً عنوان تب مرورگر (*Title*)

آموزش تغییر عنوان مرورگر با *useEffect* (تعامل با *DOM*)

در ری‌اکت، وظیفه اصلی ما مدیریت رابط کاربری (*UI*) است. اما گاهی نیاز داریم با دنیای بیرون از ری‌اکت ارتباط برقرار کنیم. به این کار *Side Effect* می‌گوییم. یکی از رایج‌ترین نمونه‌های آن، تغییر دادن عنوان تب مرورگر است.

مرحله ۱: ساخت کامپوننت مدیریت عنوان

یک فایل جدید در مسیر *src/components/TitleManager.jsx* بسازید. در این تمرین، می‌خواهیم کاری کنیم که با تایپ کردن کاربر در یک *Input*، عنوان تب مرورگر همزمان عوض شود.

کد فایل *src/components/TitleManager.jsx*:

```
import React, { useState, useEffect } from 'react';

const TitleManager = () => {
  const [name, setName] = useState('');

  // برای تغییر عنوان صفحه useEffect استفاده از
  useEffect(() => {
    // برای تغییر عنوان تب DOM دسترسی مستقیم به
    document.title = name ? `سلام ${name}` : "به آموزش ری‌اکت خوش آمدید";

    // نکته آموزشی: این کد خارج از ری‌اکت اجرا می‌شود
    console.log("عنوان مرورگر به روز شد");

    // تغییر کند name این افکت فقط وقتی اجرا می‌شود که مقدار
  }, [name]);

  return (
    <div style={{ direction: 'rtl', padding: '۲۰px', textAlign: 'center' }}>
```

```

<h۲>نام خود را وارد کنید تا عنوان تب عوض شود</h۲>
<input
  type="text"
  value={name}
  onChange={(e) => setName(e.target.value)}
  placeholder="نام شما..."
  style={{ padding: '۱۰px', fontSize: '۱۶px', borderRadius: '۵px' }}
/>
<p style={{ marginTop: '۲۰px' }}>
  خروجی فعلی در تب مرورگر: <strong>{document.title}</strong>
</p>
</div>
);
};

export default TitleManager;

```

مرحله ۲: استفاده در *App.jsx*

حالا این کامپوننت را در فایل اصلی فراخوانی کنید.

کد فایل *src/App.jsx*:

```

import React from 'react';
import TitleManager from '../components/TitleManager';

function App() {
  return (
    <div className="App">
      <TitleManager />
    </div>
  );
}

export default App;

```

نکات کلیدی

۱. چرا از *useEffect* استفاده کردیم؟

اگر کد `document.title = name` را مستقیماً داخل بدنه تابع کامپوننت بنویسیم، هر بار که کامپوننت به هر دلیلی رندر شود، این عملیات سنگین (تعامل با *DOM*) تکرار می‌شود. *useEffect* به ما اجازه می‌دهد این رفتار را کنترل کنیم.

۲. مفهوم *Direct DOM Manipulation*

به دانشجویان توضیح دهید که ری‌اکت یک *Virtual DOM* دارد. اما *document* یا *window* بخشی از *Virtual DOM* نیستند. برای دسترسی به آن‌ها باید از "اثرات جانبی" استفاده کرد.

بکارگیری *useEffect* ساخت یک ساعت آنالوگ

ساخت یک ساعت آنالوگ با استفاده از تصاویر مجزا، یکی از بهترین تمرین‌ها برای یادگیری نحوه تعامل *useEffect* با "زمان" (*Timers*) در دنیای واقعی است.

در این آموزش، ما از *setInterval* برای به‌روزرسانی زمان در هر ثانیه استفاده می‌کنیم و با استفاده از استایل‌های *CSS* (ویژگی *transform: rotate*) عقربه‌ها را می‌چرخانیم.

مرحله ۱: ایجاد کامپوننت ساعت

یک فایل جدید به نام *AnalogClock.jsx* در پوشه *src/components* بسازید.

کد فایل *src/components/AnalogClock.jsx*

```
import React, { useState, useEffect } from 'react';

const AnalogClock = () => {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    // ۱. ایجاد یک تایمر که هر ۱۰۰۰ میلی‌ثانیه (۱ ثانیه) اجرا می‌شود
    const timer = setInterval(() => {
      setTime(new Date());
    }, ۱۰۰۰);

    // ۲. تابع پاک‌سازی (Cleanup):
    // بسیار مهم است که وقتی کامپوننت بسته می‌شود، تایمر را متوقف کنیم تا حافظه اشغال
    // نشود.
    return () => clearInterval(timer);
  }, []); // آرایه وابستگی خالی یعنی فقط یک بار در هنگام لود شدن اجرا شود
}
```

```

// محاسبه زوایای چرخش عقربه‌ها
const seconds = time.getSeconds();
const minutes = time.getMinutes();
const hours = time.getHours();

// فرمول محاسبه درجه چرخش (۳۶۰ درجه کل دایره است)
const secDegrees = (seconds / ۶۰) * ۳۶۰;
const minDegrees = ((minutes / ۶۰) * ۳۶۰) + ((seconds / ۶۰) * ۶);
const hourDegrees = ((hours % ۱۲) / ۱۲ * ۳۶۰) + ((minutes / ۶۰) * ۳۰);

// استایل مشترک برای عقربه‌ها
const handStyle = {
  position: 'absolute',
  bottom: '۵۰%',
  left: '۵۰%',
  transformOrigin: 'bottom center', // نقطه چرخش پایین عقربه (مرکز ساعت) باشد
  transition: seconds === ۰ ? 'none' : 'all ۰,۰۵s cubic-bezier(۰,۱, ۲,۷, ۰,۵۸, ۱)'
};

return (
  <div style={{ display: 'flex', justifyContent: 'center', padding: '۵۰px', direction: 'rtl' }}>
    <div style={{
      position: 'relative',
      width: '۳۰۰px',
      height: '۳۰۰px',
      backgroundImage: 'url("/Clock.jpg")', // فرض بر این است که عکس‌ها در
    }}>
      هستند public پوشه

{ /* عقربه ساعت‌شمار */ }
        

        { /* عقربه دقیقه‌شمار */ }
        


```

```

    { /* عقربه ثانیه شمار */ }
    

    { /* نقطه مرکز ساعت */ }
    <div style={{
      position: 'absolute', top: '۵۰%', left: '۵۰%', transform:
'translate(-۵۰%, -۵۰%)',
      width: '۱۲px', height: '۱۲px', backgroundColor: 'black',
borderRadius: '۵۰%', zIndex: ۶
    }}></div>
  </div>
</div>
);
};

export default AnalogClock;

```

مرحله ۲: فراخوانی در *App.jsx*

کافیست کامپوننت را در فایل اصلی وارد کنید.

```

import AnalogClock from './components/AnalogClock';

function App() {
  return (
    <div>
      <h۲ style={{ textAlign: 'center' }}>React و ساعت آنالوگ با</h۲>
      <AnalogClock />
    </div>
  );
}

```

نکات کلیدی برای آموزش به دانشجویان:

۱. محل قرارگیری تصاویر: به دانشجویان یادآوری کنید که تصاویر *Clock.jpg* و بقیه باید در پوشه *public* پروژه باشند تا با آدرس *name.jpg/* در دسترس قرار بگیرند.
۲. چرا *setInterval* در *useEffect*؟ چون ما به یک فرآیند تکرار شونده نیاز داریم که به محض بالا آمدن ساعت شروع به کار کند و مستقل از رندرینگ عادی ری‌اکت باشد.

۳. اهمیت *Cleanup*: این بخش بسیار حیاتی است. توضیح دهید که اگر *clearInterval* را ننویسیم، حتی اگر کاربر به صفحه دیگری برود، این تایمر در پس‌زمینه مرورگر باقی می‌ماند و باعث کندی سیستم (*Memory Leak*) می‌شود.

۴. ویژگی *transformOrigin*: توضیح دهید که برای چرخیدن عقربه‌ها حول مرکز ساعت، باید نقطه اتکای تصویر (*bottom center*) را ثابت نگه داریم.

فصل پنجم: مدیریت State

یادآوری *useState*:

در ری‌اکت ۱۹.۲ (و کلاً در تمام نسخه‌های اخیر ری‌اکت)، *useState* پرکاربردترین ابزاری است که با آن به کامپوننت‌های خود «حافظه» می‌دهید.

در حالت عادی، متغیرهای معمولی در ری‌اکت با هر بار رندر شدن برنامه پاک می‌شوند. اما *useState* اجازه می‌دهد یک مقدار را ذخیره کنید تا حتی وقتی صفحه دوباره چیده می‌شود، آن مقدار باقی بماند.

ساختار ساده *useState*

تصور کنید می‌خواهیم یک شمارنده (*Counter*) بسازیم:

```
const [count, setCount] = useState(0);
```

این خط کد سه بخش اصلی دارد:

۱. *count*: نام متغیری که مقدار فعلی را نگه می‌دارد (مثلاً عدد ۰).

۲. *setCount*: تابعی که از آن برای تغییر دادن مقدار استفاده می‌کنیم.

۳. *useState(0)*: مقدار اولیه را داخل پرانتز می‌نویسیم.

یک مثال کامل و واقعی

در این مثال، هر بار که روی دکمه کلیک کنید، عدد یک واحد اضافه می‌شود:

```
import { useState } from 'react';
```

```
function MyCounter() {
  // تعریف استیت با مقدار اولیه ۰
  const [count, setCount] = useState(۰);

  return (
    <div>
      <p>شما {count} بار کلیک کرده‌اید </p>

      { /* استفاده کنیم setCount برای تغییر مقدار، حتما باید از */ }
      <button onClick={() => setCount(count + ۱)}>
        اضافه کردن
      </button>
    </div>
  );
}
```

چرا نمی‌توانیم از متغیرهای معمولی استفاده کنیم؟

اگر بنویسید $let\ x = 0$ و بعد بگویید $x = x + 1$ اتفاقی در ظاهر سایت نمی‌افتد. چون ری‌اکت نمی‌فهمد که چیزی تغییر کرده است.

اما وقتی از **setCount** استفاده می‌کنید، ری‌اکت دو کار انجام می‌دهد:

- مقدار جدید را در حافظه ذخیره می‌کند.
- اتوماتیک صفحه را دوباره رندر می‌کند تا کاربر تغییر را ببیند.

چند نکته طلایی:

- آپدیت در ری‌اکت ۱۹: در نسخه‌های جدید، مدیریت تغییرات استیت بهینه‌تر شده است، اما ساختار استفاده از آن دقیقاً مشابه قبل است.
- تغییر ناپذیری: شما نباید مستقیماً بنویسید $count = 5$. همیشه باید از تابع دوم (مثل **setCount**) استفاده کنید.

تابع پاکسازی (*Cleanup Function*):

برای جلوگیری از *Memory Leak* (نشت حافظه)، کارهایی مثل بستن تایمرها یا لغو اشتراک‌ها را اینجا انجام می‌دهیم:

```
useEffect(() => {
  const timer = setInterval(() => console.log('تیک تاک!'));
  return () => clearInterval(timer); // پاکسازی هنگام حذف کامپوننت
}, []);
```

۲. هوک‌های کاربردی دیگر

الف) *useRef*: دسترسی به *DOM* و مقادیر تغییرپذیر

از این هوک زمانی استفاده می‌کنیم که بخواهیم به یک المان (مثل *Input*) مستقیماً دستور بدهیم (مثلاً فوکوس کردن) یا مقداری را ذخیره کنیم که با تغییرش، کامپوننت دوباره رندر نشود.

```
const inputRef = useRef(null);
const handleFocus = () => {
  inputRef.current.focus(); // فوکوس مستقیم روی اینپوت
};
return <input ref={inputRef} />;
```

هوک *useRef* مثل یک «دستگیره» یا «نشانگر» عمل می‌کند.

در حالت عادی، *React* خودش همه چیز را مدیریت می‌کند و شما به کدهای پشت صحنه (*DOM*) کاری ندارید. اما گاهی لازم است مستقیماً به یک المان (مثل یک جعبه متن یا اینپوت) دستور بدهید.

این کد چه کار می‌کند؟

```
const inputRef = useRef(null);
```

یک دستگیره خالی می‌سازد (مثل اینکه یک برچسب آماده کنید اما هنوز روی چیزی نچسبانده باشید).

```
<input ref={inputRef} />
```

این برچسب را به اینپوت مورد نظر می‌چسباند. حالا *React* می‌داند که *inputRef* دقیقاً به همین ورودی اشاره دارد.

```
inputRef.current.focus();
```

کلمه *current* یعنی «همین الان برچسب روی چه چیزی است؟». با این دستور، انگار با انگشت به آن اینپوت اشاره می‌کنید و می‌گویید: «کلیک کن اینجا (فوکوس کن)!»

یک مثال در دنیای واقعی:

فرض کنید یک فرم طولانی دارید و می‌خواهید وقتی کاربر روی دکمه "ویرایش" کلیک کرد، مکان‌نما (*Cursor*) خودبه‌خود برود داخل جعبه متن "نام" تا کاربر معطل نشود. اینجا است که از *useRef* استفاده می‌کنید.

خلاصه:

- *useState*: برای وقتی است که می‌خواهید چیزی را روی صفحه تغییر دهید (رندر مجدد).
- *useRef*: برای وقتی است که می‌خواهید به یک المان دستور بدهید (بدون تغییر کل صفحه).

۲- سایر کامپوننت‌ها

(ب) *useContext*: مدیریت وضعیت سراسری

برای اینکه یک داده (مثل تم شب/روز یا اطلاعات کاربر) را بدون پاس دادن لایه به لایه (*Prop Drilling*) به همه فرزندان برسانیم.

```
// 1. ایجاد Context
const ThemeContext = createContext('light');

// 2. استفاده در کامپوننت فرزند
const theme = useContext(ThemeContext);
```

(ج) *useReducer*: مدیریت وضعیت پیچیده

وقتی منطق *State* شما پیچیده می‌شود (مثلاً چندین عمل مختلف روی یک لیست)، *useReducer* جایگزین بهتری برای *useState* است. این هوک شبیه به *Redux* عمل می‌کند.

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
};
const [state, dispatch] = useReducer(reducer, { count: 0 });
// استفاده: dispatch({ type: 'increment' })
```

ایجاد *Todolist* با کمک *useReducer*

فرض کنید یک پوشه به نام *components* دارید و فایلی به نام *TodoItem.jsx* در آن است.

۱. فایل *App.jsx* (مدیریت مرکزی)

در اینجا منطق اصلی برنامه (*Reducer*) را می‌نویسیم.

```
import React, { useReducer } from 'react';
import TodoItem from '../components/TodoItem'; // components فرض بر وجود پوشه

// ۱. تابع Reducer: تصمیم‌گیرنده اصلی برای تغییرات
const todoReducer = (state, action) => {
  switch (action.type) {
    case 'ADD_TODO':
      return [...state, { id: Date.now(), text: action.payload }];
    case 'DELETE_TODO':
      return state.filter(todo => todo.id !== action.payload);
    default:
      return state;
  }
};

function App() {
  // ۲. مقدار اولیه یک آرایه خالی است: useReducer تعریف
  const [todos, dispatch] = useReducer(todoReducer, []);

  const handleAdd = (e) => {
    if (e.key === 'Enter' && e.target.value) {
      // ۳. ارسال دستور (dispatch) به Reducer
      dispatch({ type: 'ADD_TODO', payload: e.target.value });
      e.target.value = '';
    }
  };

  return (
    <div style={{ padding: '۲۰px' }}>
      <h۲>لیست کارهای من</h۲>
      <input onKeyDown={handleAdd} placeholder="... کار جدید را بنویس و اینتر بزن" />
      <ul>
        {todos.map(todo => (
          /* ۴. فراخوانی کامپوننت فرزند */
          <TodoItem key={todo.id} todo={todo} dispatch={dispatch} />
        ))}
      </ul>
    </div>
  );
}
```

```

    </div>
  );
}

export default App;

```

۲. فایل `components/ToDoItem.jsx` (کامپوننت فرزند)

اینجا فقط وظیفه نمایش و ارسال دستور حذف را داریم.

```

import React from 'react';

function ToDoItem({ todo, dispatch }) {
  return (
    <li style={{ margin: '10px 0' }}>
      {todo.text}
      <button
        onClick={() => dispatch({ type: 'DELETE_TODO', payload: todo.id })}
        style={{ marginRight: '10px', color: 'red' }}
      >
        حذف
      </button>
    </li>
  );
}

export default ToDoItem;

```

توضیح ساده این کد:

۱. **reducer** (آشپزخانه): جایی است که می‌گوییم "اگر دستور اضافه کردن آمد، این کار را بکن" و "اگر دستور حذف آمد، آن کار را بکن".

۲. **dispatch** (گارسون): تنها راه ما برای صحبت با *Reducer* است. ما مستقیماً لیست را تغییر نمی‌دهیم، بلکه به دیسپچ می‌گوییم: «این دستور را ببر به آشپزخانه (*Reducer*)».

۳. **action** (سفارش): همان چیزی است که به دیسپچ می‌دهیم. شامل یک **type** (نوع کار) و معمولاً یک **payload** (داده مورد نیاز مثل متن کار یا *ID*) است.

مزیت بزرگ: اگر برنامه شما بزرگ شود و بخواهید قابلیت‌های "ویرایش"، "انجام شد" یا "پاک کردن همه" را اضافه کنید، فقط کافیست چند *case* جدید به *todoReducer* اضافه کنید و ظاهر برنامه (*UI*) اصلاً پیچیده نمی‌شود.

مثال عملی: ترکیب هوک‌ها در یک کامپوننت

در این مثال، از **useEffect** برای تغییر عنوان صفحه و از **useRef** برای فوکوس خودکار استفاده شده است:

```
import React, { useState, useEffect, useRef } from 'react';

function AdvancedCounter() {
  const [count, setCount] = useState(0);
  const inputRef = useRef(null);

  // تغییر عنوان صفحه با هر بار تغییر count
  useEffect(() => {
    document.title = `تعداد: ${count}`;

    // برمی‌گردانیم cleanup یا تایمری داشتیم، تابع Listener نکته: اگر اینجا
    }, [count]);

  // هندلر برای افزایش مقدار با استفاده از حالت تابعی
  const handleIncrement = () => {
    setCount(prevCount => prevCount + 1);
  };

  // هندلر برای فوکوس روی اینپوت
  const focusInput = () => {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  };

  return (
    <div style={{ padding: '۲۰px', fontFamily: 'Tahoma, sans-serif',
      textAlign: 'right', direction: 'rtl' }}>
      <h۲>شمارنده هوشمند</h۲>
      <p>عدد فعلی: <strong>{count}</strong></p>
      <button onClick={handleIncrement}>افزایش مقدار</button>

      <hr style={{ margin: '۲۰px ۰' }} />

      <input
        ref={inputRef}
        placeholder="چیزی تایپ کنید..."
        style={{ marginLeft: '۱۰px', padding: '۰px' }}
      />
    </div>
  );
}
```

```

        <button onClick={focusInput}>فوکوس روی اینپوت</button>
      </div>
    );
  }

export default AdvancedCounter;

```

۱. وارد کردن ابزارها (*Imports*)

```
import React, { useState, useEffect, useRef } from 'react';
```

- ***useState***: برای ذخیره داده‌هایی که تغییر می‌کنند و نیاز است در صفحه نمایش داده شوند (مثل عدد شمارنده).
- ***useEffect***: برای انجام کارهایی که مستقیماً به رندر شدن مربوط نیستند (مثل تغییر عنوان تب مرورگر یا ارتباط با سرور).
- ***useRef***: برای دسترسی مستقیم به یک المان **HTML** بدون اینکه صفحه دوباره رندر شود.

۲. تعریف وضعیت (*State*)

```
const [count, setCount] = useState(0);
```

- این خط یک متغیر به نام ***count*** با مقدار اولیه ۰ می‌سازد.
- ***setCount*** تابعی است که هر وقت بخواهیم عدد را عوض کنیم، از آن استفاده می‌کنیم. با هر بار فراخوانی این تابع، **React** می‌فهمد که باید ظاهر صفحه را بروزرسانی کند.

۳. تعریف مرجع (*Ref*)

```
const inputRef = useRef(null);
```

- این یک "قلاب" یا "چنگک" ایجاد می‌کند که قرار است به ***input*** ما متصل شود. مقدار اولیه آن را ***null*** می‌گذاریم چون هنوز المان در صفحه ساخته نشده است.

۴. مدیریت اثرات جانبی (*useEffect*)

```
useEffect(() => {
```

```
document.title = `تعداد: ${count}`;  
}, [count]);
```

- این کد می‌گوید: "هر وقت مقدار *count* تغییر کرد، عنوان تب مرورگر را بروزرسانی کن."
- **[count]**: این آرایه وابستگی (*Dependency Array*) است. اگر آن را خالی بگذاریم، کد فقط یک‌بار (موقع باز شدن صفحه) اجرا می‌شود. اما چون *count* را داخلش گذاشتیم، با هر بار کلیک روی دکمه، این کد دوباره اجرا می‌شود.

۵. تابع افزایش عدد (*Logic*)

```
const handleIncrement = () => {  
  setCount(prevCount => prevCount + ۱);  
};
```

- در اینجا از *Functional Update* استفاده کردیم. *prevCount* یعنی "آخرین مقداری که همین الان در حافظه هست". این روش بسیار امن‌تر از *setCount(count + ۱)* است، مخصوصاً در برنامه‌های پیچیده.

۶. تابع فوکوس (*DOM Interaction*)

```
const focusInput = () => {  
  if (inputRef.current) {  
    inputRef.current.focus();  
  }  
};
```

- این تابع از طریق *inputRef.current* مستقیماً به خودِ المان *<input>* در دنیای *HTML* دسترسی پیدا کرده و متد *focus()* را روی آن اجرا می‌کند.

۷. بخش نمایش (*JSX*)

```
<p>عدد فعلی: <strong>{count}</strong></p>
```

- هر جا که بخواهیم مقدار متغیرها را در *HTML* نمایش دهیم، آن‌ها را داخل *{ }* می‌گذاریم.

```
<input ref={inputRef} ... />
```

- با نوشتن *ref={inputRef}* ما آن چنگکی که در مرحله ۳ ساختیم را به این اینپوت متصل کردیم. حالا هر تغییری در این اینپوت از طریق *inputRef* قابل دسترسی است.

مثال ایجاد تم شب

سلام دوستان! امروز می‌خواهیم یاد بگیریم چطور یک سیستم تعویض تم (حالت شب و روز) حرفه‌ای با استفاده از *Context API* بسازیم.

مرحله اول: ویرایش فایل اصلی (*App.jsx*)

ابتدا کد زیر را در فایل *src/App.jsx* قرار دهید. این فایل نقطه شروع برنامه ماست که بقیه اجزا را در بر می‌گیرد.

```
/* src/App.jsx */
import React from 'react';
import { ThemeProvider } from './compotents/ThemeContext';
import Navbar from './compotents/Navbar';
import MainContent from './compotents/MainContent';
import './App.css';

function App() {
  return (
    <ThemeProvider>
      <div className="app-layout">
        <Navbar />
        <div className="content-area">
          <MainContent />
        </div>
      </div>
    </ThemeProvider>
  );
}

export default App;
```

توضیح کد:

- *ThemeProvider*: این کامپوننت مثل یک چتر کل برنامه ما را پوشش می‌دهد تا تمام اجزای داخلی (مثل *Navbar* و *MainContent*) به وضعیت تم دسترسی داشته باشند.
- *app-layout*: یک کلاس کلی برای استایل‌دهی به ساختار اصلی صفحه است.

مرحله دوم: ایجاد پوشه کامپوننت‌ها

اگر هنوز پوشه‌ای به نام *compotents* (یا *components*) در پوشه *src* خود ندارید، همین الان آن را بسازید. ما تمام فایل‌های زیر را در این پوشه قرار می‌دهیم.

مرحله سوم: قلب تپنده برنامه (*ThemeContext.jsx*)

یک فایل به نام *ThemeContext.jsx* بسازید و کد زیر را در آن قرار دهید. این فایل منطق تغییر تم را مدیریت می‌کند.

```
import React, { createContext, useState, useEffect, useContext } from
'react';

const ThemeContext = createContext();

export const ThemeProvider = ({ children }) => {
  const getInitialTheme = () => {
    if (typeof window === 'undefined') return 'light';
    const savedTheme = localStorage.getItem('app-theme');
    if (savedTheme) return savedTheme;
    const prefersDark = window.matchMedia('(prefers-color-scheme:
dark)').matches;
    return prefersDark ? 'dark' : 'light';
  };

  const [theme, setTheme] = useState(getInitialTheme());

  useEffect(() => {
    document.documentElement.setAttribute('data-theme', theme);
    localStorage.setItem('app-theme', theme);
  }, [theme]);

  const toggleTheme = () => {
    setTheme((prevTheme) => (prevTheme === 'light' ? 'dark' : 'light'));
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

export const useTheme = () => useContext(ThemeContext);
```

توضیح کد:

- *createContext*: یک فضای اشتراکی برای داده‌ها ایجاد می‌کند.

- ***getInitialTheme***: بررسی می‌کند که آیا کاربر قبلاً تمی انتخاب کرده (در *localStorage*) یا اینکه تم سیستم‌عامل کاربر چیست.
- ***useEffect***: هر بار که تم تغییر می‌کند، یک ویژگی به نام *data-theme* به تگ *HTML* اصلی اضافه می‌کند تا *CSS* بفهمد چه رنگ‌هایی را نمایش دهد.
- ***useTheme***: یک *Hook* اختصاصی است تا بقیه کامپوننت‌ها خیلی راحت از تم استفاده کنند.

مرحله چهارم: نوار ناوبری (*Navbar.jsx*)

فایل *Navbar.jsx* را بسازید. این بخش در بالای سایت قرار می‌گیرد.

```
import React from 'react';
import { useTheme } from '../ThemeContext';
import ThemeToggle from '../ThemeToggle';
import './Navbar.css';

const Navbar = () => {
  return (
    <nav className="navbar">
      <div className="nav-brand">پنل دانشجویی ریکت</div>
      <ul className="nav-links">
        <li><a href="#home">صفحه اصلی</a></li>
        <li><a href="#courses">دروس من</a></li>
      </ul>
      <div className="nav-actions">
        <ThemeToggle />
      </div>
    </nav>
  );
};
```

توضیح کد:

- ما در اینجا دکمه تغییر تم (*ThemeToggle*) را داخل نوار ابزار قرار داده‌ایم تا همیشه در دسترس باشد.

مرحله پنجم: دکمه سوئیچ (*ThemeToggle.jsx*)

این همان دکمه‌ای است که کاربر روی آن کلیک می‌کند تا تم عوض شود.

```
import React from 'react';
import { useTheme } from '../ThemeContext';
import './ThemeToggle.css';
```

```
const ThemeToggle = () => {
  const { theme, toggleTheme } = useTheme();

  return (
    <button onClick={toggleTheme} className="theme-btn">
      {theme === 'light' ? '🌞 حالت روز' : '🌙 حالت شب'}
    </button>
  );
};
```

توضیح کد:

- **toggleTheme**: تابعی است که از **Context** گرفتیم و وظیفه‌اش معکوس کردن تم فعلی است.
- شرط ساده: اگر تم روشن بود آیکون ماه و اگر تاریک بود آیکون خورشید را نشان می‌دهیم.

مرحله ششم: محتوای اصلی (**MainContent.jsx**)

در نهایت محتوایی که تغییرات را نمایش می‌دهد:

```
import React from 'react';
import { useTheme } from './ThemeContext';

const MainContent = () => {
  const { theme } = useTheme();

  return (
    <main className="main-container">
      <div className="card">
        <h1>خوش آمدید</h1>
        <p>وضعیت فعلی سیستم: <strong>{theme === 'light' ? 'روز' : 'شب'}</strong></p>
      </div>
    </main>
  );
};
```

توضیح کد:

- در اینجا فقط با خواندن مقدار **theme** از **Context**، متن را به صورت داینامیک فارسی‌سازی کردیم.

زیباسازی و استایل‌دهی (**CSS**)

دوستان عزیز، برای اینکه تم‌های ما (تاریک و روشن) واقعاً کار کنند، از **CSS Variables** استفاده می‌کنیم. ابتدا فایل‌های **CSS** مربوط به هر بخش را در همان پوشه **compotents** ایجاد کنید.

۱. استایل نوار ناوبری (**Navbar.css**)

یک فایل به نام **Navbar.css** بسازید و کدهای زیر را در آن قرار دهید:

```
/* src/compotents/Navbar.css */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.8rem 2rem;
  background-color: var(--nav-bg);
  color: var(--text-color);
  border-bottom: 1px solid var(--border-color);
  transition: all var(--transition-speed) ease;
}

.nav-brand {
  font-size: 1.5rem;
  font-weight: bold;
  color: var(--primary-color);
}

.nav-links {
  display: flex;
  list-style: none;
  gap: 20px;
}

.nav-links a {
  text-decoration: none;
  color: var(--text-color);
  font-size: 1rem;
  opacity: 0.8;
}

.nav-links a:hover {
  opacity: 1;
  color: var(--primary-color);
}
```

توضیح: در اینجا از متغیرهایی مثل **var(--nav-bg)** استفاده کردیم. این یعنی رنگ پس‌زمینه نوار ابزار، بسته به اینکه تم در چه حالتی باشد، به صورت خودکار تغییر می‌کند.

۲. استایل محتوای اصلی (*MainContent.css*)

فایل *MainContent.css* را ایجاد کرده و کدهای زیر را کپی کنید:

```
/* src/compotents/MainContent.css */
.main-container {
  display: flex;
  justify-content: center;
  align-items: center;
  padding: ۵۰px ۲۰px;
  min-height: ۶۰vh;
}

.card {
  background-color: var(--card-bg);
  color: var(--text-color);
  padding: ۳۰px;
  border-radius: ۱۵px;
  border: ۱px solid var(--border-color);
  box-shadow: ۰ ۴px ۱۰px rgba(۰, ۰, ۰, ۰.۰۵);
  text-align: center;
  max-width: ۵۰۰px;
  transition: all var(--transition-speed) ease;
}

.card h۱ {
  color: var(--primary-color);
  margin-bottom: ۲۰px;
}
```

توضیح: ما برای کارت وسط صفحه، از **transition** استفاده کردیم تا وقتی تم عوض می‌شود، تغییر رنگ‌ها به نرمی و با انیمیشن انجام شود.

۳. استایل دکمه تغییر تم (*ThemeToggle.css*)

در نهایت فایل *ThemeToggle.css* را برای دکمه بسازید:

```
/* src/compotents/ThemeToggle.css */
.toggle-wrapper {
  display: flex;
  justify-content: center;
  margin: ۲۰px;
}

.theme-btn {
  display: flex;
}
```

```

align-items: center;
gap: ۱۰px;
padding: ۱۰px ۲۰px;
border-radius: ۲۰px;
border: ۲px solid var(--border-color);
background-color: var(--card-bg);
color: var(--text-color);
cursor: pointer;
transition: all ۰,۳s ease;
font-weight: ۶۰۰;
}

.theme-btn:hover {
  transform: scale(۱,۰۵);
  box-shadow: ۰ ۴px ۱۵px rgba(۰,۰,۰,۰,۱);
}

.icon {
  font-size: ۱,۲rem;
}

```

توضیح: این دکمه دارای افکت *hover* است که هنگام نگه داشتن ماوس، کمی بزرگتر می‌شود تا تجربه کاربری بهتری (*UX*) ایجاد کند.

دوستان عزیز، حالا نوبت به فایل *src/index.css* می‌رسد. این فایل همان‌جایی است که ما به مرورگر می‌گوییم وقتی تم *"dark"* است چه رنگ‌هایی را جایگزین رنگ‌های *"light"* کند.

لطفاً کدهای زیر را در فایل *index.css* جایگزین کنید:

```

/* src/index.css */

/* تعریف تم پایه (روز) */
:root {
  --bg-color: #ffffff;
  --text-color: #۱a۱a۱a;
  --primary-color: #۳۴۹۸db;
  --nav-bg: #f۸f۹fa;
  --card-bg: #ffffff;
  --border-color: #dddddd;
  --transition-speed: ۰,۳s;
}

/* تعریف تغییرات تم شب */
[data-theme="dark"] {
  --bg-color: #۱۲۱۲۱۲;
  --text-color: #f۰f۰f۰;
  --primary-color: #۲۹۸۰b۹;
  --nav-bg: #۱e۱e۱e;
  --card-bg: #۱e۱e۱e;
}

```

```

--border-color: #۳۳۳۳۳۳;
}

/* اعمال متغیرها به بدنه سند */
body {
  background-color: var(--bg-color);
  color: var(--text-color);
  transition: background-color var(--transition-speed) ease, color var(--transition-speed) ease;
  margin: ۰;
  padding: ۰;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, sans-serif;
}

#root {
  max-width: ۱۲۸۰px;
  margin: ۰ auto;
  padding: ۲rem;
  text-align: center;
}

```

توضیح کد (بسیار مهم):

- **root:** در اینجا متغیرهای پیش فرض (حالت روز) را تعریف کردیم. هر جای پروژه که از `var(--bg-color)` استفاده کنیم، رنگ سفید اعمال می شود.
- `ThemeContext.jsx` در فایل `Selector` هوشمند است. وقتی در `["data-theme="dark"]` این یک `Selector` هوشمند است. وقتی در فایل `ThemeContext.jsx` یادتان هست که نوشتیم `document.documentElement.setAttribute('data-theme', theme)` با این کار، وقتی تم به `"dark"` تغییر می کند، مرورگر به صورت خودکار مقادیر جدید (رنگ های تیره) را جایگزین مقادیر قبلی می کند.
- **transition در Body:** این خط باعث می شود وقتی دکمه را می زنید، رنگ کل صفحه به آرامی و با یک انیمیشن نرم تغییر کند، نه به صورت ناگهانی که چشم کاربر را اذیت کند.

فصل ششم: مسیریابی (Routing) با React Router

هدف: ساخت برنامه های *Single Page Application (SPA)* با قابلیت مسیریابی.

۱. [نصب و راه اندازی React Router DOM](#)

۲. ایجاد Route های اصلی

- * کامپوننت های `Route`، `Routes`، `BrowserRouter`
- * تعریف مسیرها و ارتباط آن ها با کامپوننت ها

۳. مسیریابی یویا

- * پارامترهای داینامیک در `URL (:id)`
- * هوک `useParams` برای دسترسی به پارامترها
- ۴. لینک دهی و `Navigation`
- * استفاده از `Link` به جای تگ `<a>`
- * هوک `useNavigate` برای هدایت برنامه ای (`Programmatic Navigation`)

نصب و راه اندازی React Router DOM

برای استفاده از مسیریابی در `React`، باید کتابخانه `react-router-dom` را نصب کنید:

```
npm install react-router-dom
```

- `react-router-dom` مخصوص وب است. اگر پروژه `React Native` دارید، از `react-navigation`

استفاده می کنید.

- همیشه مطمئن شوید نسخه کتابخانه با نسخه `React` شما سازگار باشد.

ایجاد روت های اصلی

ایجاد ساختار فایل ها

برای این آموزش، سه صفحه ساده ایجاد می کنیم `Home`، `About`، `Contact`.

۱. داخل فولدر `src`، یک فولدر به نام `pages` بسازید:

```
src/
├── pages/
│   ├── Home.jsx
│   ├── About.jsx
│   └── Contact.jsx
```

۲. محتوای هر صفحه را ایجاد کنید:

```
//Home.jsx
function Home() {
```

```
    return <h۲>صفحه خانه</h۲>;  
  }  
  
  export default Home;
```

```
//About.jsx  
function About() {  
  return <h۲>درباره ما</h۲>;  
}  
  
export default About;
```

```
//Contact.jsx  
  
function Contact() {  
  return <h۲>تماس با ما</h۲>;  
}  
  
export default Contact;
```

مرحله ۴: تنظیم Router در *App.js*

حالا باید مسیرها را به پروژه اضافه کنیم.

۱. فایل *App.jsx* را باز کنید.

۲. کد زیر را جایگزین محتویات آن کنید:

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";  
import Home from "../pages/Home";  
import About from "../pages/About";  
import Contact from "../pages/Contact";  
  
function App() {  
  return (  
    <BrowserRouter>  
      <nav style={{ marginBottom: "۲۰px" }}>  
        <Link to="/" style={{ marginRight: "۱۰px" }}>خانه</Link>  
        <Link to="/about" style={{ marginRight: "۱۰px" }}>درباره ما</Link>  
        <Link to="/contact">تماس با ما</Link>  
      </nav>  
      <Routes>  
        <Route path="/" element={Home} />  
        <Route path="/about" element={About} />  
        <Route path="/contact" element={Contact} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```



```

</nav>

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
</BrowserRouter>
);
}

export default App;

```

توضیحات آموزشی:

- **BrowserRouter** روت اصلی که مسیرها را مدیریت می‌کند.
- **Routes** مجموعه مسیرها.
- **Route** مسیر جداگانه با **path** و کامپوننت **element**.
- **Link** لینک داخلی برای جابجایی بین صفحات بدون رفرش.
- می‌توانید استایل ساده برای **Navbar** اضافه کنید تا ظاهر لینک‌ها بهتر شود.

توضیح خط به خط کدهای بالا

وارد کردن کتابخانه‌ها و کامپوننت‌ها

```

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "../pages/Home";
import About from "../pages/About";
import Contact from "../pages/Contact";

```

توضیح:

۱. **BrowserRouter**

- کامپوننت اصلی مدیریت مسیرها در **SPA** است.
- کل برنامه را درون آن می‌گذاریم تا مسیرها بدون رفرش صفحه کنترل شوند.
- شبیه یک "موتور مسیریابی" عمل می‌کند.

۲. **Routes**

- مجموعه‌ای از مسیرها را در خود نگه می‌دارد.

- داخل آن، همه **<Route>** ها قرار می گیرند.

۳. **Route**

- یک مسیر جداگانه تعریف می کند.

- دو ویژگی مهم دارد:

▪ **path** آدرس **URL** →

▪ **element** کامپوننتی که هنگام باز شدن این مسیر نمایش داده می شود.

۴. **Link**

- برای ایجاد لینک داخلی به مسیرهای مختلف استفاده می شود.

- تفاوت با **<a>**: صفحه رفرش نمی شود و **SPA** باقی می ماند.

۵. وارد کردن کامپوننت های صفحات (**Home, About, Contact**)

- این ها فایل های جداگانه هستند که در فولدر **pages** ساخته شده اند.

- هر کدام یک صفحه ساده با محتوای متفاوت نمایش می دهند.

۳. تعریف کامپوننت **App**

```
function App() {  
  return (  
    ...  
  );  
}
```

- کامپوننت اصلی برنامه.

- همان جایی است که **Router** و مسیرها را تنظیم می کنیم.

۴. استفاده از **BrowserRouter**

```
<BrowserRouter>  
  ...  
</BrowserRouter>
```

- همه مسیرها و لینک ها باید داخل **BrowserRouter** باشند.

- این کامپوننت **URL** مرورگر را رصد می کند و مطابق آن کامپوننت مناسب را نمایش می دهد.

۵. ایجاد نوار ناوبری (Navbar)

```
<nav style={{ marginBottom: "۲۰px" }}>
  <Link to="/" style={{ marginRight: "۱۰px" }}>خانه</Link>
  <Link to="/about" style={{ marginRight: "۱۰px" }}>درباره ما</Link>
  <Link to="/contact">تماس با ما</Link>
</nav>
```

توضیح:

- **<nav>**: یک بخش ناوبری ساده برای لینک‌ها.
 - **<Link to="...">**: مسیر مقصد را مشخص می‌کند.
 - مثال **<Link to="/about">**: درباره ما **</Link>** وقتی کلیک شود، **URL** به **about** تغییر می‌کند و کامپوننت **About** نمایش داده می‌شود.
 - استایل داخلی:
 - **marginBottom: "20px"** → فاصله پایین **Navbar** تا محتوا
 - **marginRight: "10px"** → فاصله بین لینک‌ها
- نکته مهم: استفاده از **Link** به جای **<a>** باعث می‌شود صفحه رفرش نشود و تجربه **SPA** حفظ شود.

۶. تعریف مسیرها با **Routes** و **Route**

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
```

توضیح خط به خط:

۱. **<Routes>**

- مجموعه مسیرها را نگه می‌دارد.

- **React Router** بررسی می‌کند **URL** فعلی با کدام **path** مطابقت دارد و فقط آن **Route** را نمایش می‌دهد.

```
<Route path="/" element={<Home />} />
```

- وقتی آدرس مرورگر **/** است، کامپوننت **<Home />** نمایش داده می‌شود.

```
<Route path="/about" element={<About />} />
```

- وقتی آدرس صفحه **/about** است، کامپوننت **About** نمایش داده می‌شود.

```
<Route path="/contact" element={<Contact />} />
```

- وقتی آدرس صفحه برابر با **/contact** است، کامپوننت **Contact** نمایش داده می‌شود.

○

۷. خروجی و نتیجه نهایی

```
npm start
```

- وقتی پروژه را اجرا می‌کنیم: (**npm start**)
 - لینک‌ها در **Navbar** ظاهر می‌شوند.
 - با کلیک روی هر لینک، **URL** تغییر می‌کند و بدون رفرش صفحه، صفحه مرتبط نمایش داده می‌شود.

- نتیجه: یک **SPA** ساده با سه صفحه و ناوبری داخلی آماده است.

نکات آموزشی مهم

۱. همیشه مسیر **/** را اولین **Route** قرار دهید تا تطابق درست انجام شود.
۲. اگر مسیر داینامیک داشتید، مثل **/product/:id**، باید از **useParams** برای خواندن پارامتر استفاده کنید.
۳. **BrowserRouter** باید در ریشه **App** باشد.
۴. از **Link** به جای **<a>** استفاده کنید تا **SPA** حفظ شود.

مرحله ۶: تمرین برای دانشجویان

۱. یک صفحه جدید به نام **Services.js** بسازید و مسیر **/services** اضافه کنید.
۲. از **Link** برای هدایت به این مسیر استفاده کنید.
۳. یک مسیر داینامیک **/product/:id** ایجاد کنید و با **useParams** مقدار **id** را نمایش دهید.

۳. مسیریابی پویا

هدف:

- ایجاد مسیرهایی که بخشی از **URL** آن‌ها قابل تغییر باشد (پارامتر داینامیک).
- استفاده از **useParams** برای دسترسی به مقادیر داینامیک در کامپوننت.

پیش‌نیازها

- نصب: **React Router DOM**

```
npm install react-router-dom
```

- پروژه خام **React** آماده باشد (مثلاً با **create-react-app**).

۲. ایجاد ساختار فایل‌ها

فرض می‌کنیم یک پروژه با مسیرهای محصولات داریم.

در فولدر **src**، فولدر **pages** را بسازید و فایل‌های زیر را اضافه کنید:

```
src/
├── pages/
│   ├── Products.jsx
│   └── ProductDetail.jsx
```

۴. ایجاد صفحه **Products**

```
// src/pages/Products.jsx
import { Link } from "react-router-dom";

const products = [
  { id: ۱, name: "محصول ۱" },
  { id: ۲, name: "محصول ۲" },
  { id: ۳, name: "محصول ۳" }
];
```

```
function Products() {
  return (
    <div>
      <h2>لیست محصولات</h2>
      <ul>
        {products.map(product => (
          <li key={product.id}>
            <Link to={` /products/${product.id}`}>{product.name}</Link>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default Products;
```

توضیح:

- یک آرایه از محصولات ساخته ایم.
- برای هر محصول، یک لینک به مسیر *products/:id* ایجاد شده است.
- *Link* باعث می شود که صفحه رفرش نشود و *SPA* باقی بماند.

۵. ایجاد صفحه *ProductDetail*

```
// src/pages/ProductDetail.js
import { useParams } from "react-router-dom";

function ProductDetail() {
  const { id } = useParams(); // خواندن پارامتر داینامیک
  return (
    <div>
      <h2>جزئیات محصول</h2>
      <p>شناسه محصول: {id}</p>
    </div>
  );
}

export default ProductDetail;
```

توضیح:

- *useParams* یک هوک است که پارامترهای داینامیک *URL* را برمی گرداند.

- `{id}` مقدار پارامتر `id` را می‌گیرد و می‌توان در صفحه نمایش داد یا برای گرفتن داده از سرور استفاده کرد.

۶. اضافه کردن مسیرهای داینامیک در `App.js`

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Products from "../pages/Products";
import ProductDetail from "../pages/ProductDetail";

function App() {
  return (
    <BrowserRouter>
      <nav style={{ marginBottom: "۲۰px" }}>
        <Link to="/products" style={{ marginRight: "۱۰px" }}>محصولات</Link>
      </nav>

      <Routes>
        <Route path="/products" element={<Products />} />
        <Route path="/products/:id" element={<ProductDetail />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

توضیح:

۱. مسیر `→ products` صفحه لیست محصولات
۲. مسیر `→ products/:id` صفحه جزئیات محصول با پارامتر داینامیک
۳. وقتی روی محصول کلیک می‌کنیم:
 - `URL` مثال `/products/2` :
 - `ProductDetail` نمایش داده می‌شود و مقدار `id` برابر ۲ خواهد بود.

۶. نتیجه نهایی

- کاربران روی هر محصول کلیک کنند، به صفحه جزئیات همان محصول هدایت می‌شوند.
- بدون رفرش کل صفحه، `SPA` حفظ می‌شود.

- می‌توان با همان **id** درخواست به سرور داد یا داده محصول را نمایش داد.

۷. تمرین برای دانشجویان

۱. آرایه محصولات را با نام، توضیح و قیمت گسترش دهید.
۲. در صفحه **ProductDetail**، اطلاعات کامل محصول را بر اساس **id** نمایش دهید.
۳. مسیر **products/:id/:color** اضافه کنید و از دو پارامتر داینامیک استفاده کنید.

لینک‌دهی و Navigation

هدف:

- ایجاد لینک داخلی بین صفحات با **<Link>** بدون رفرش صفحه.
- هدایت برنامه‌ای (**Programmatic Navigation**) با هوک **useNavigate** برای مواقعی که نیاز داریم کاربر پس از انجام یک عمل مثل فرم یا کلیک، به مسیر دیگری برود.

پیش‌نیازها

- پروژه **React** با **React Router** آماده باشد.
- فایل‌ها و مسیرها (**Pages**) مانند **Home, About, Contact** یا مثال قبلی **Products** و **ProductDetail** موجود باشد.
- کتابخانه **react-router-dom** نصب باشد:

```
npm install react-router-dom
```

۲. استفاده از **Link** به جای **<a>**

در **React SPA**، نباید از **** استفاده کنیم، چون باعث رفرش کل صفحه می‌شود و تجربه **SPA** را خراب می‌کند.

مثال عملی

```
// src/components/Navbar.jsx
import { Link } from "react-router-dom";
```



```
function Navbar() {
  return (
    <nav style={{ marginBottom: "۲۰px" }}>
      <Link to="/" style={{ marginRight: "۱۰px" }}>خانه</Link>
      <Link to="/about" style={{ marginRight: "۱۰px" }}>درباره ما</Link>
      <Link to="/contact">تماس با ما</Link>
    </nav>
  );
}

export default Navbar;
```

توضیح:

- $to="/" \rightarrow$ مسیر مقصد
- **Link** بدون رفرش صفحه، مسیر را تغییر می‌دهد.
- استایل ساده برای فاصله بین لینک‌ها اضافه شده.
- می‌توانید این **Navbar** را در **App.js** وارد کنید:

```
import Navbar from "../components/Navbar";
...
<BrowserRouter>
  <Navbar />
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
  </Routes>
</BrowserRouter>
```

با این کار، لینک‌ها **SPA** را حفظ می‌کنند و جابجایی بین صفحات سریع و روان است.

۳. هدایت برنامه‌ای (**Programmatic Navigation**) با **useNavigate**

گاهی نیاز داریم از طریق کد کاربر را به مسیر دیگری هدایت کنیم، مثلاً بعد از ثبت فرم یا کلیک دکمه.

مثال عملی: فرم ورود

```
// src/pages/Login.jsx
import { useNavigate } from "react-router-dom";
```

```
function Login() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // فرض کنیم ورود موفق است
    alert("ورود موفقیت آمیز بود");
    navigate("/dashboard"); // هدایت برنامه‌ای به صفحه داشبورد
  };

  return (
    <div>
      <h۲>فرم ورود</h۲>
      <button onClick={handleLogin}>ورود</button>
    </div>
  );
}

export default Login;
```

توضیح:

۱. `useNavigate()` یک هوک **React Router** است که تابعی برای هدایت برمی‌گرداند.

۲. `navigate("/dashboard")` → کاربر به مسیر `/dashboard` هدایت می‌شود.

۳. می‌توانید از مقادیر داینامیک هم استفاده کنید:

```
navigate(`/profile/${username}`);
```

۴. اضافه کردن مسیرها در `App.js`

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "../pages/Home";
import About from "../pages/About";
import Contact from "../pages/Contact";
import Login from "../pages/Login";
import Dashboard from "../pages/Dashboard";
import Navbar from "../components/Navbar";

function App() {
  return (
    <BrowserRouter>
      <Navbar />

      <Routes>
```

```

    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
    <Route path="/login" element={<Login />} />
    <Route path="/dashboard" element={<Dashboard />} />
  </Routes>
</BrowserRouter>
);
}

export default App;

```

- مسیر **login** → فرم ورود

- مسیر **dashboard** → صفحه مقصد بعد از هدایت برنامه‌ای

۵. جمع‌بندی آموزشی

- **Link:** برای لینک‌دهی داخلی بین صفحات **SPA** استفاده می‌شود.
- **useNavigate:** برای هدایت برنامه‌ای (کلیک دکمه، بعد از ارسال فرم، یا بعد از عملیات موفق) استفاده می‌شود.
- با ترکیب این دو، برنامه شما هم تجربه **SPA** واقعی دارد و هم کنترل کامل روی مسیرها خواهید داشت.

۶. تمرین برای دانشجویان

۱. در صفحه **Products**، بعد از کلیک روی یک محصول، از **useNavigate** برای هدایت به **products/:id** استفاده کنید.
۲. دکمه **Logout** بسازید که با **useNavigate** کاربر را به صفحه **Login** هدایت کند.
۳. تمرین کنید که لینک‌ها استایل متفاوت داشته باشند وقتی صفحه فعلی همان مسیر است (**active link**).

فصل هفتم: کار با فرم‌ها و مدیریت آن‌ها

هدف: ایجاد فرم‌های کنترل شده و اعتبارسنجی آن‌ها.

۱. فرم‌های کنترل شده (*Controlled Forms*)
۲. مدیریت چندین فیلد ورودی به صورت پویا
۳. ارسال داده فرم و مدیریت وضعیت آن
۴. اعتبارسنجی سمت کلاینت (*Client-Side Validation*)
۵. معرفی کتابخانه‌های معتبرسازی مانند *React Hook Form* (به عنوان گزینه حرفه‌ای)

فصل هشتم: بهینه‌سازی و ساخت (*Build*) (۱ جلسه)

هدف: بهینه‌سازی برنامه برای تولید و آماده‌سازی برای دیپلوی.

۱. بهینه‌سازی *Performance*
 - * هوک‌های *useMemo* و *useCallback*
 - * *React.memo* برای بهینه‌سازی رندر کامپوننت‌ها
۲. ساخت نهایی با *Vite*
 - * آنالیز کردن فایل *build* خروجی
 - * پیش‌نمایش نسخه *Production* با *npm run preview*

فصل نهم: تست‌نویسی (*Testing*) (۱ جلسه)

هدف: آشنایی با اصول تست‌نویسی برای کامپوننت‌های *React*.

۱. معرفی *Vitest* و *Testing Library*

۲. نوشتن اولین تست برای یک کامپوننت ساده

۳. تست تعاملات کاربر (*Fire Event*)

فصل دهم: پروژه نهایی و جمع‌بندی (۲ جلسه)

هدف: استفاده از تمامی مفاهیم آموزش داده شده در یک پروژه واقعی.

۱. پیاده‌سازی یک پروژه کاربردی (مثلاً برنامه مدیریت وظایف، فروشگاه کوچک یا وبلاگ)

۲. جمع‌بندی کلی مباحث دوره

۳. معرفی مسیرهای ادامه یادگیری (*Next.js, Remix, State Management Libraries* و ...)